



DEPARTMENT OF COMPUTER ENGINEERING (DCE)
HWSW CODESIGN
MMH: HSCD446164

Group: 15

Name:

Họ và tên	Mã số sinh viên
Nguyễn Thành Đô	23119138
Nguyễn Tuấn Kiệt	23119157
Lê Đoàn Thái Khương	23119155
Hà Quang Huy	23119147
Nguyễn Phi Hùng	23119152

LAB 4 IP CREATION

Lab 4 hướng dẫn quy trình tạo và tích hợp IP Core tùy chỉnh trên nền tảng Zynq SoC bằng nhiều phương pháp khác nhau gồm HDL, MathWorks HDL Coder và Vivado HLS. Sinh viên sẽ làm quen với giao tiếp AXI-Lite, đóng gói IP và kết nối IP với hệ thống Zynq trong Vivado IP Integrator.

Mục tiêu bài Lab

- + Hiểu quy trình tạo và đóng gói IP Core tương thích AXI-Lite.
- + Thiết kế IP bằng HDL, Simulink HDL Coder và Vivado HLS.
- + Kết nối IP với Zynq Processing System thông qua AXI Interface.
- + Tạo ứng dụng điều khiển phần cứng từ phía PS.
- + Làm quen với quy trình Hardware/Software Co-design trên SoC FPGA.

Công cụ cần thiết

Phần cứng

- + ZedBoard Zynq-7020

Phần mềm

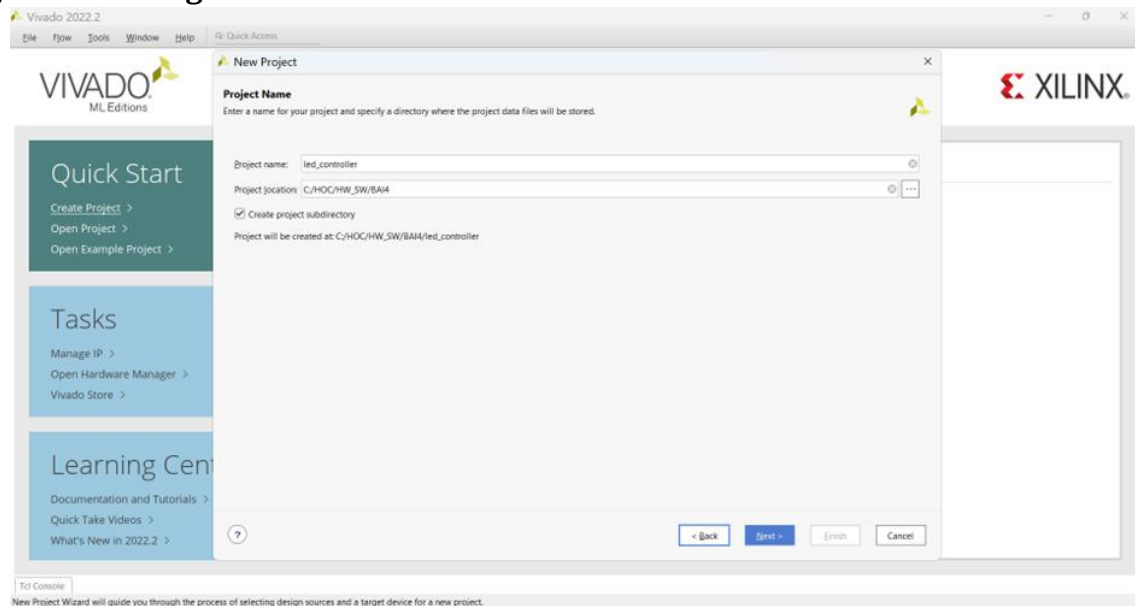
- + Xilinx Vivado
- + Vivado HLS
- + MATLAB
- + Simulink
- + HDL Coder
- + Xilinx SDK

Kết quả đạt được

- + Tạo và đóng gói thành công custom IP trên Zynq SoC.
- + Tích hợp IP vào Vivado IP Integrator.
- + Điều khiển phần cứng FPGA từ phần mềm chạy trên ARM Processor.
- + Sinh HDL tự động từ Simulink và C/C++.
- + Hiểu quy trình phát triển hệ thống nhúng FPGA SoC hiện đại.

Exercise 4A: Creating IP in HDL

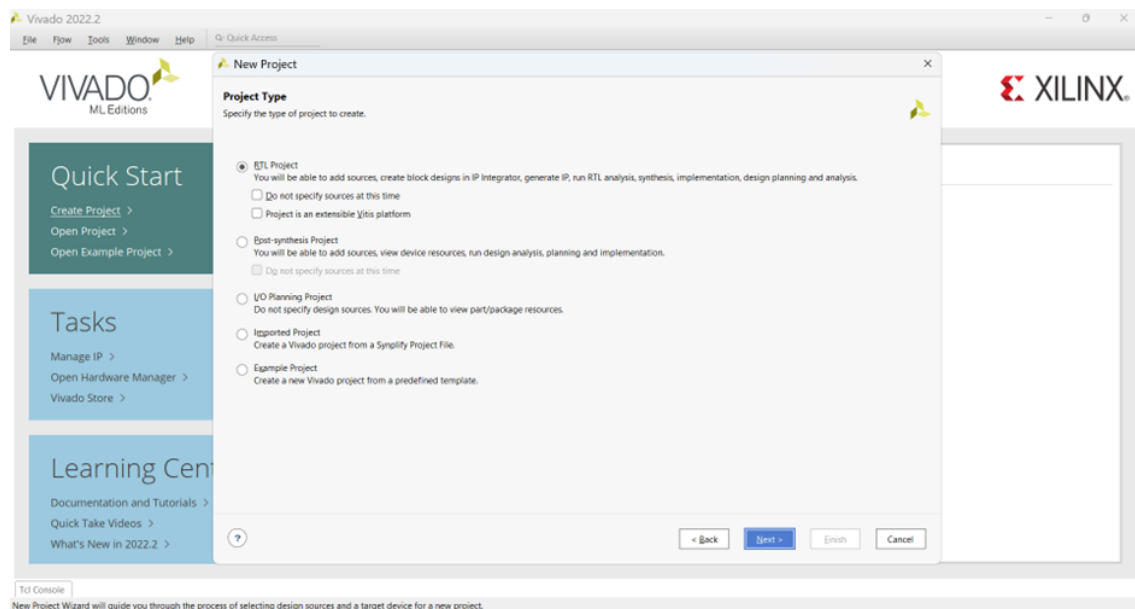
Tạo project mới trong Vivado.



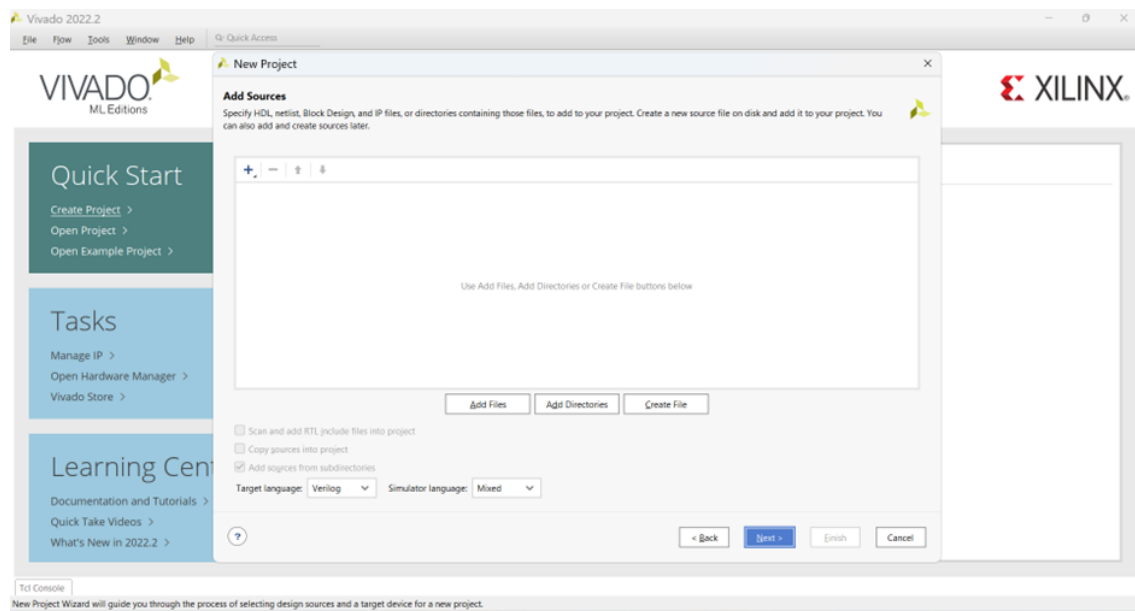
Hình 1 Tạo project mới

Đặt tên cho project là led_controller và tạo đường dẫn cho project.

Thực hiện mở Vivado, chọn Create New Project, đặt tên project là led_controller, chọn vị trí lưu và bật tùy chọn Create project subdirectory. Sau đó chọn loại project là RTL Project, ngôn ngữ thiết kế là Verilog, không thêm source, IP hay constraint ở giai đoạn này.

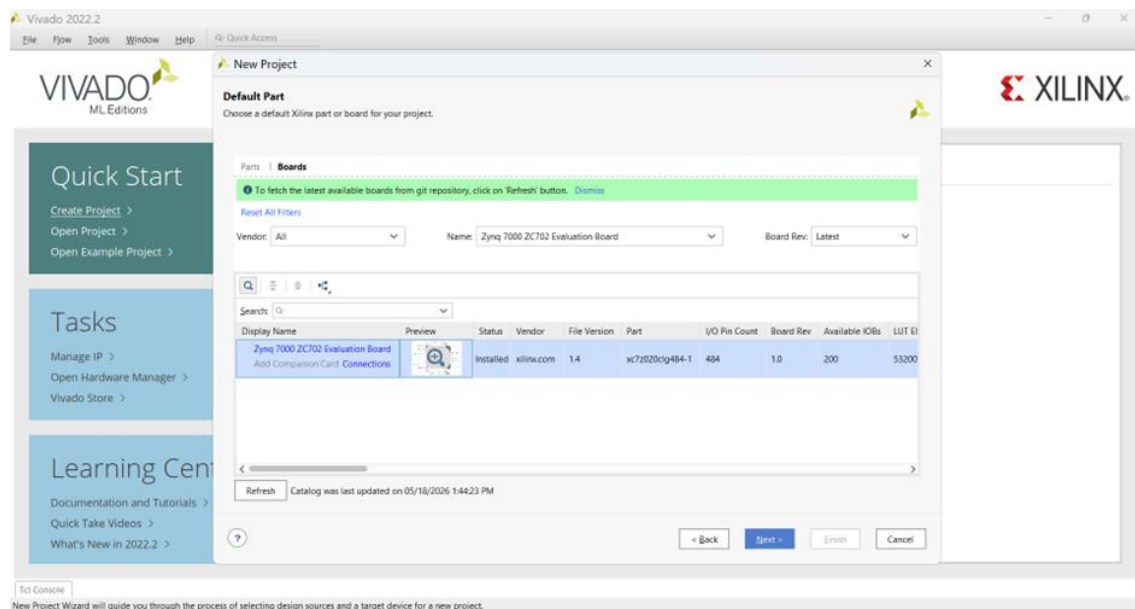


Hình 2 Chọn loại project



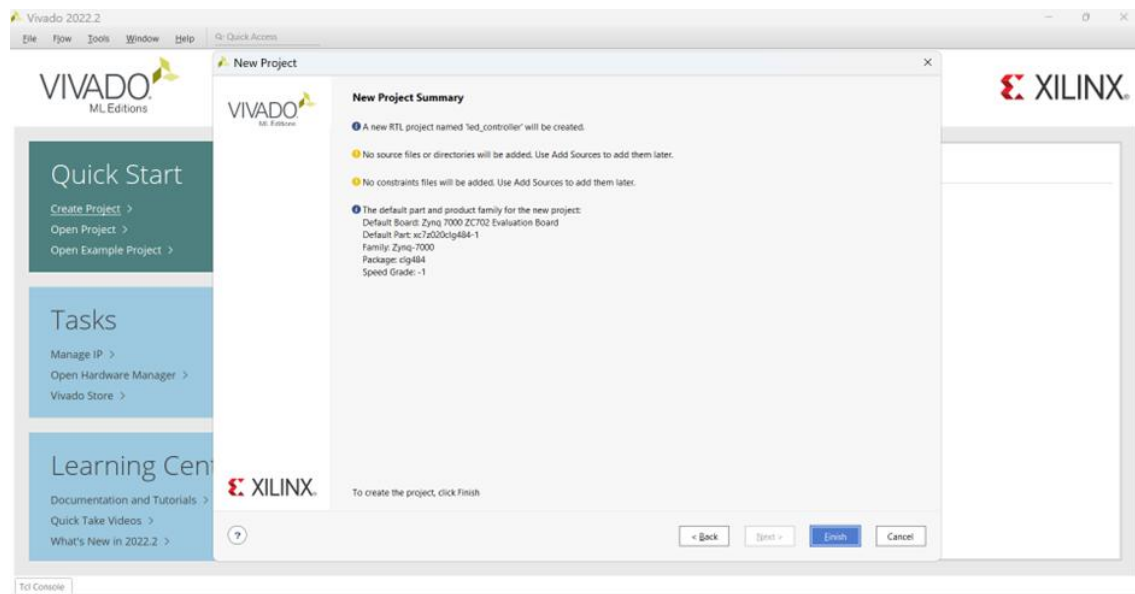
Hình 3 Add source và chọn ngôn ngữ chính

Ở phần chọn board, chọn AMD Zynq™ 7000 SoC ZC702 Evaluation Kit.



Hình 4 Chọn Board

Ý nghĩa của bước này là tạo một môi trường thiết kế phần cứng ban đầu trong Vivado. Việc chọn đúng board giúp Vivado tự nhận đúng chip Zynq-7020, chân kết nối và các thông số phần cứng cần thiết cho các bước thiết kế sau.

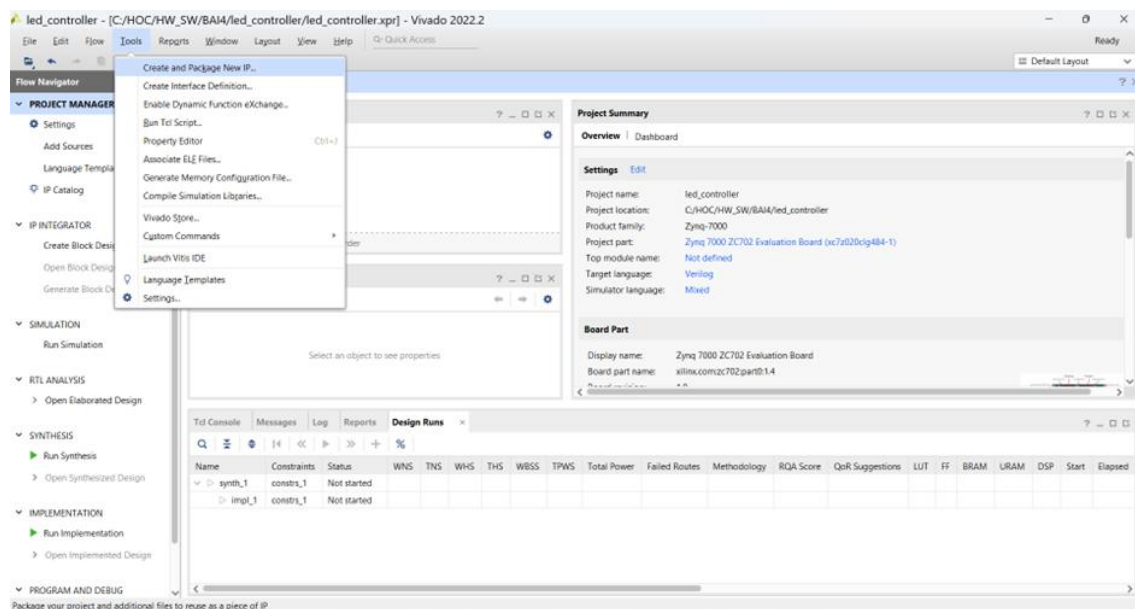


Hình 5 Hoàn thành tạo project

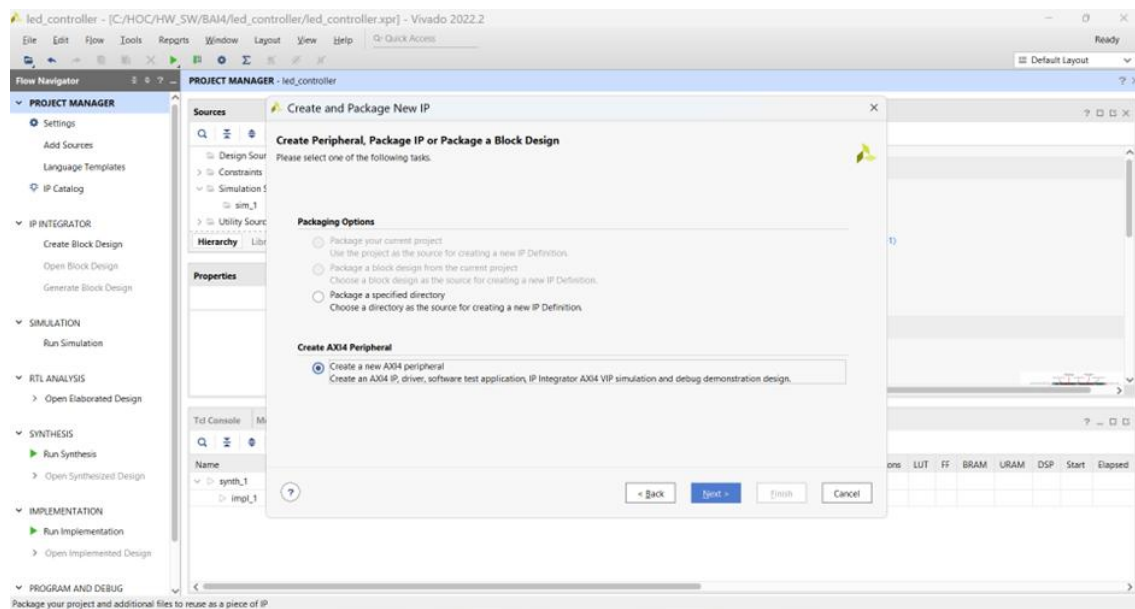
Tạo AXI Peripheral bằng Create and Package IP Wizard.

Sau khi tạo project, tiếp tục tạo một IP ngoại vi mới có giao tiếp AXI để PS có thể điều khiển phần cứng trong PL.

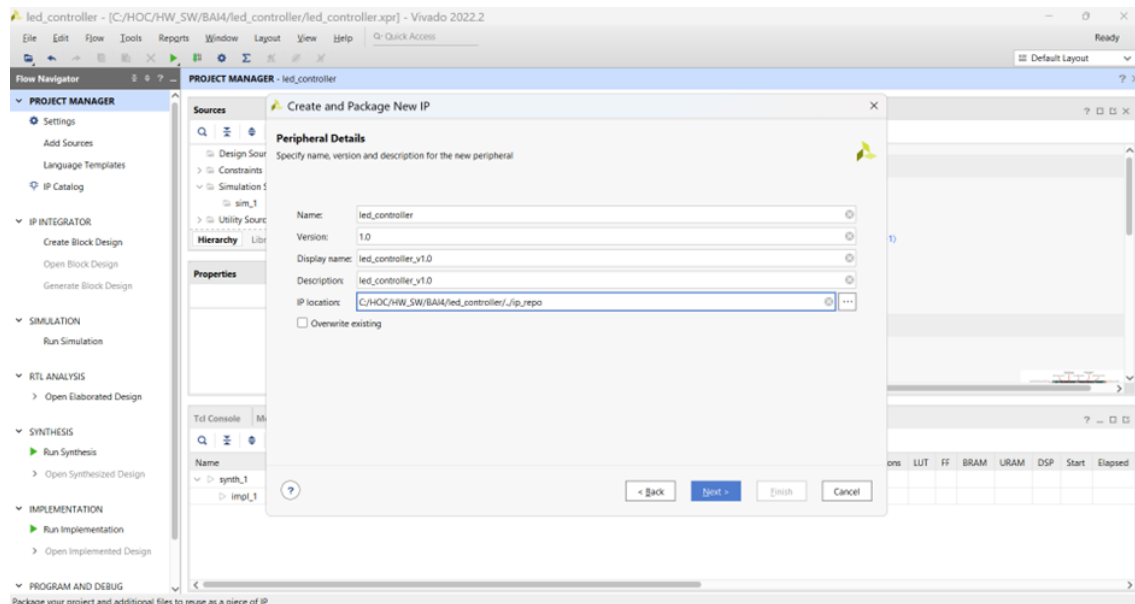
Vào Tools > Create and Package IP, chọn Create new AXI peripheral. Đặt tên IP là led_controller, phiên bản 1.0. Ở phần giao tiếp, chọn AXI4-Lite, chế độ Slave, độ rộng dữ liệu 32 bit, số thanh ghi là 4.



Hình 6 Create and Package IP Wizard

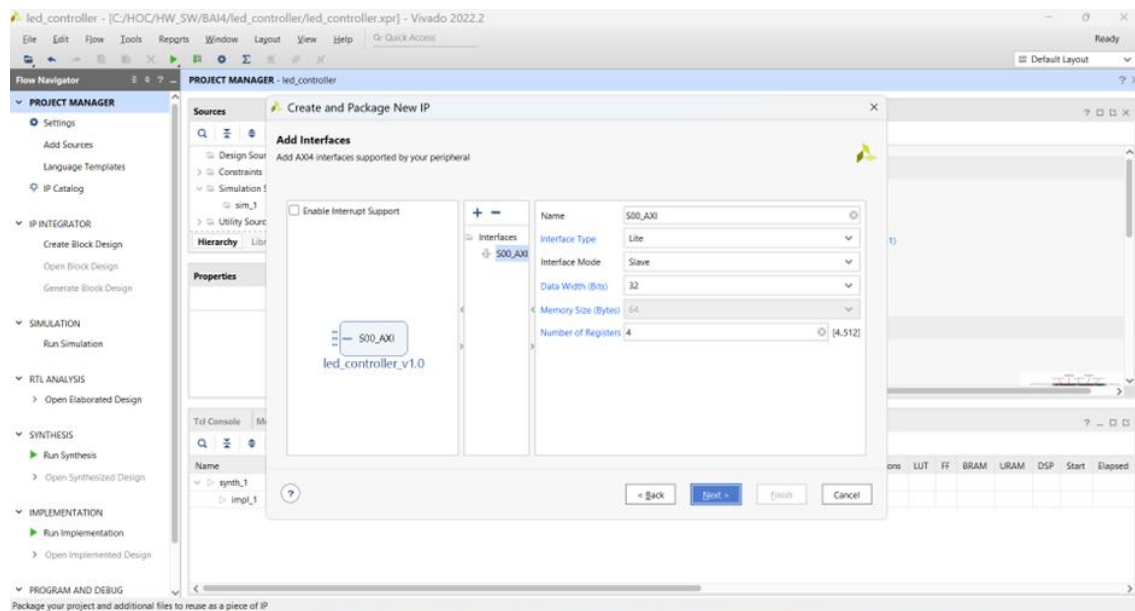


Hình 7 Create AXI4 Peripheral

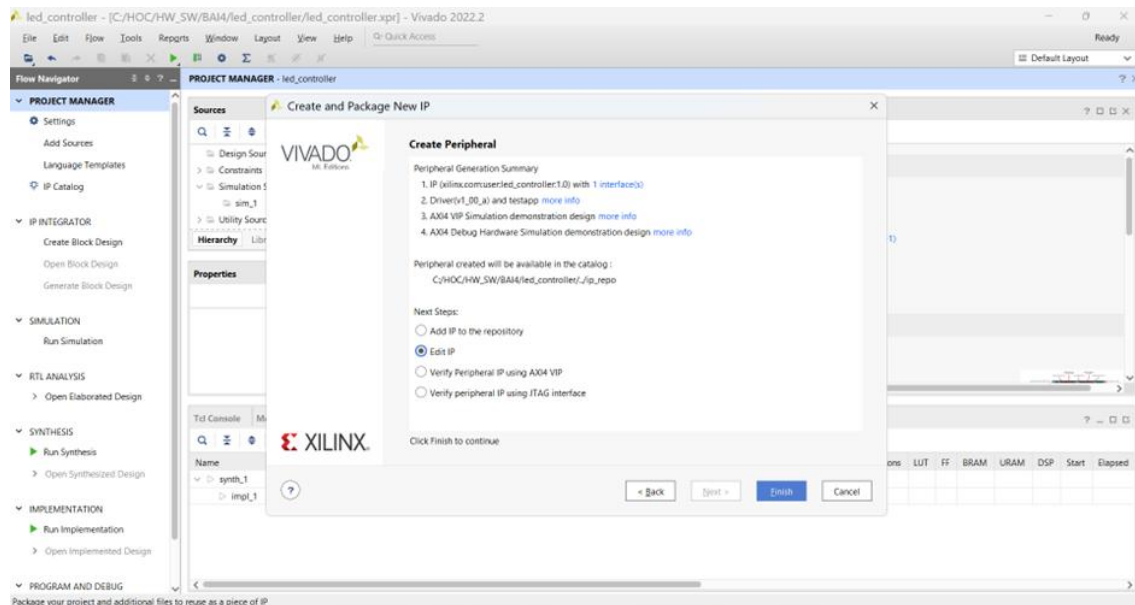


Hình 8 Cấu hình chi tiết cho IP

Ở phần giao tiếp, chọn AXI4-Lite, chế độ Slave, độ rộng dữ liệu 32 bit, số thanh ghi là 4.



Hình 9 Add Interface dialogue



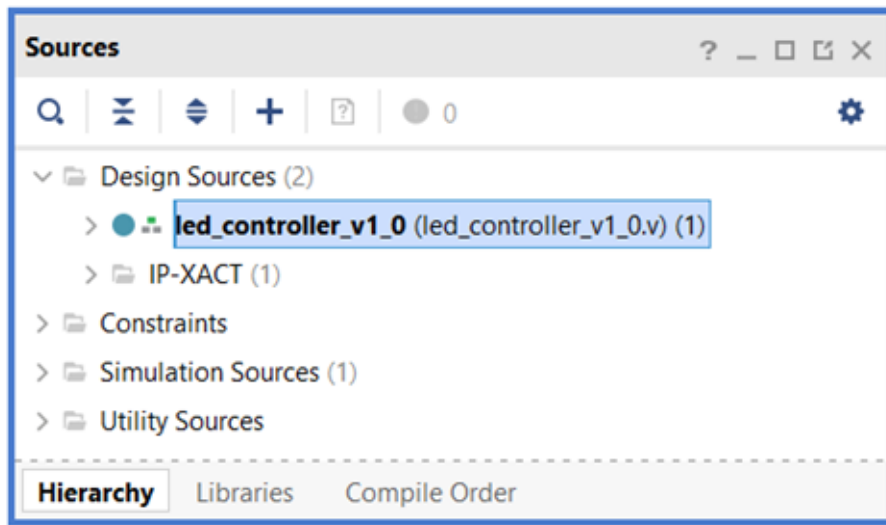
Hình 10 Create Peripheral dialogue

Chọn Edit IP. Thao tác này sẽ tạo các tệp ngoại vi IP và tạo một dự án Vivado mới, nơi bạn có thể sửa đổi chức năng của thiết bị ngoại vi trong mã nguồn HDL, sau đó đóng gói lại.

Bước này chủ yếu tạo khung IP có sẵn giao tiếp AXI4-Lite. AXI4-Lite phù hợp với thiết kế điều khiển LED vì chỉ cần truyền các giá trị điều khiển đơn giản từ PS sang PL, không cần truyền dữ liệu tốc độ cao.

Chỉnh sửa file led_controller_v1_0_S00_AXI.v.

Sau khi tạo IP, cần thêm cổng xuất dữ liệu LED vào file AXI slave để giá trị từ thanh ghi AXI có thể đưa ra LED.



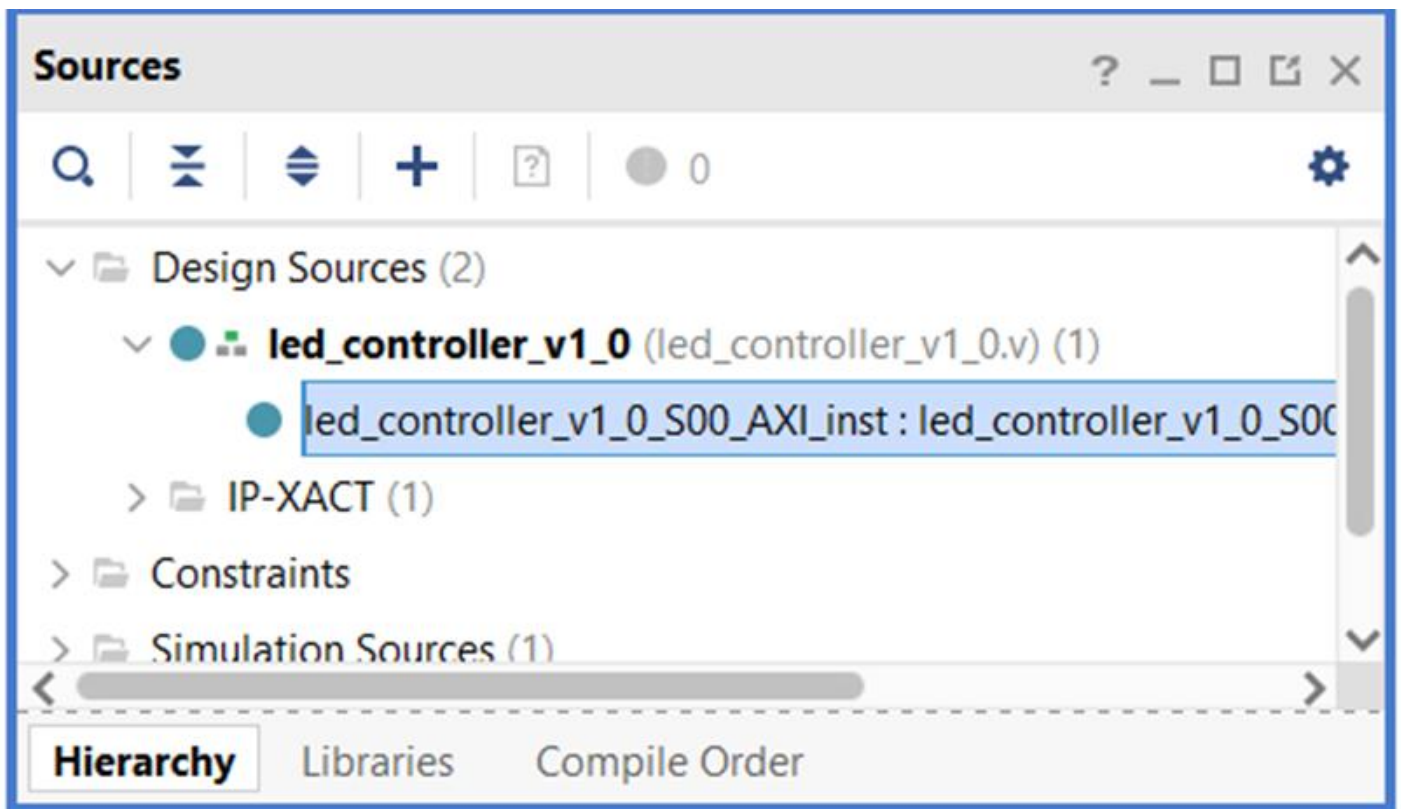
Hình 11 Mở file led_controller_v1_0.v

```

1
2 `timescale 1 ns / 1 ps
3
4 module led_controller_v1_0 #
5 (
6     // Users to add parameters here
7
8     // User parameters ends
9     // Do not modify the parameters beyond this line
10
11
12     // Parameters of Axi Slave Bus Interface S00_AXI
13     parameter integer C_S00_AXI_DATA_WIDTH  = 32,
14     parameter integer C_S00_AXI_ADDR_WIDTH  = 4
15 )
16 (
17     // Users to add ports here
18     output [7:0] LEDs_out,
19     // User ports ends
20     // Do not modify the ports beyond this line

```

Hình 12 Thêm code cho phần ports



Hình 13 File con của file led_controller_v1_0.v

Mở file lên và thêm các dòng code tại các vị trí như sau:

```

1
2 `timescale 1 ns / 1 ps
3
4 module led_controller_v1_0_S00_AXI #
5 (
6     // Users to add parameters here
7
8     // User parameters ends
9     // Do not modify the parameters beyond this line
10
11     // Width of S_AXI data bus
12     parameter integer C_S_AXI_DATA_WIDTH    = 32,
13     // Width of S_AXI address bus
14     parameter integer C_S_AXI_ADDR_WIDTH    = 4
15 )
16 (
17     // Users to add ports here
18     output wire [7:0] LEDs_out,
19     // User ports ends
20     // Do not modify the ports beyond this line

```

Hình 14 Thêm code 1

```

402 // User logic ends
403 assign LEDs_out = slv_reg0[7:0];
404 endmodule

```

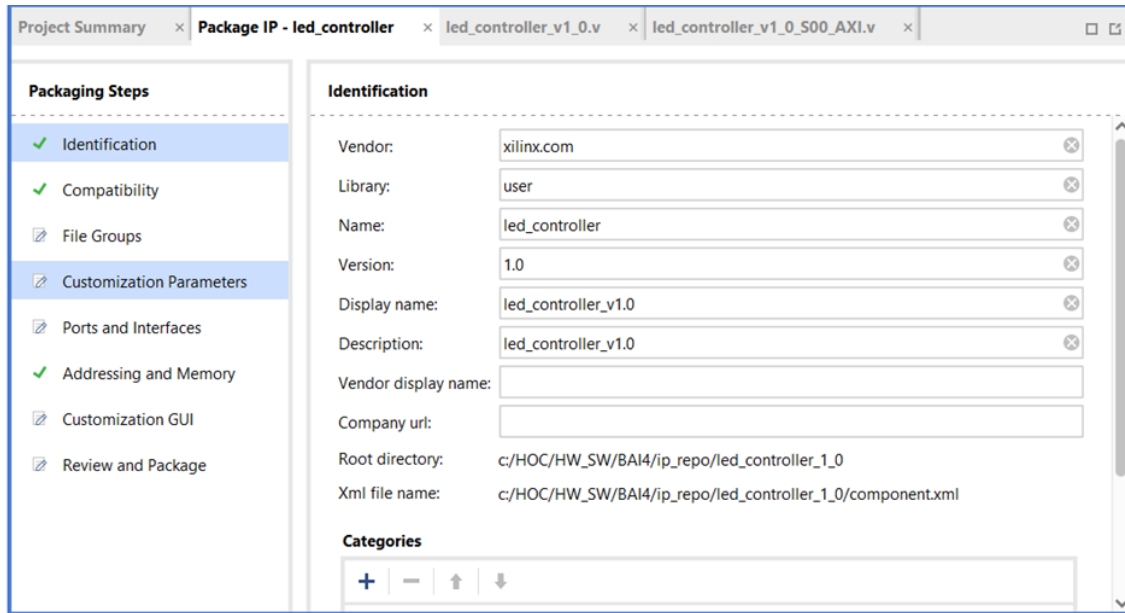
Hình 15 Thêm code 2

Ý nghĩa của bước này là tạo cổng ngõ ra 8 bit cho IP, tương ứng với 8 LED trên board. Thanh ghi slv_reg0 nhận dữ liệu từ PS thông qua AXI, sau đó 8 bit thấp của thanh ghi này được đưa trực tiếp ra cổng LEDs_out

để điều khiển trạng thái LED.

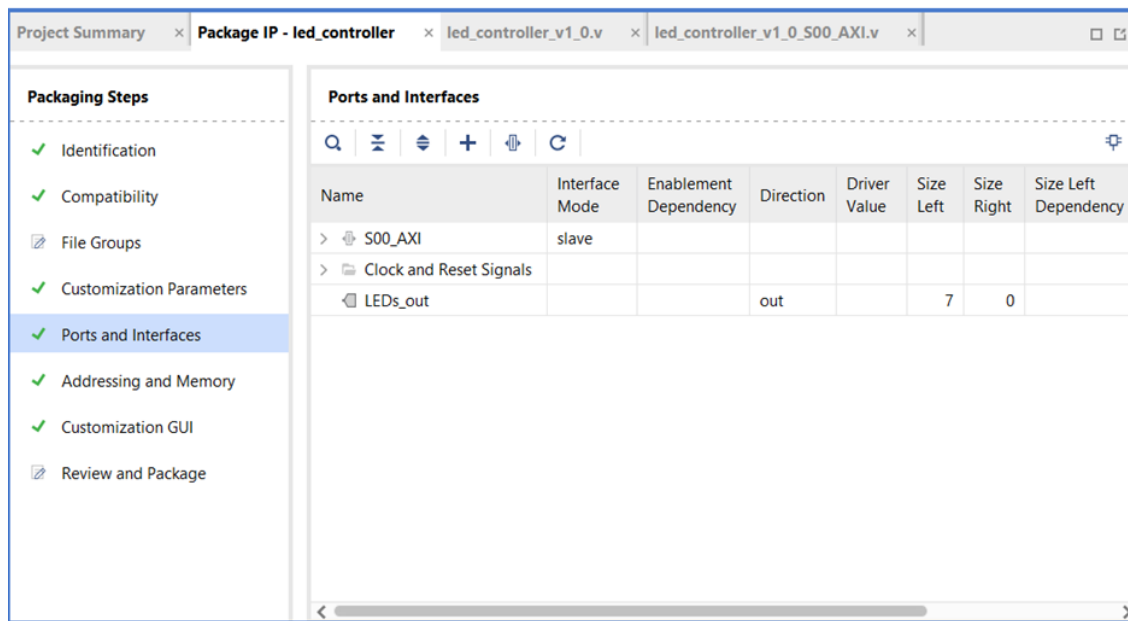
Re-package IP

Sau khi chỉnh sửa source Verilog, cần đóng gói lại IP để Vivado cập nhật các thay đổi.



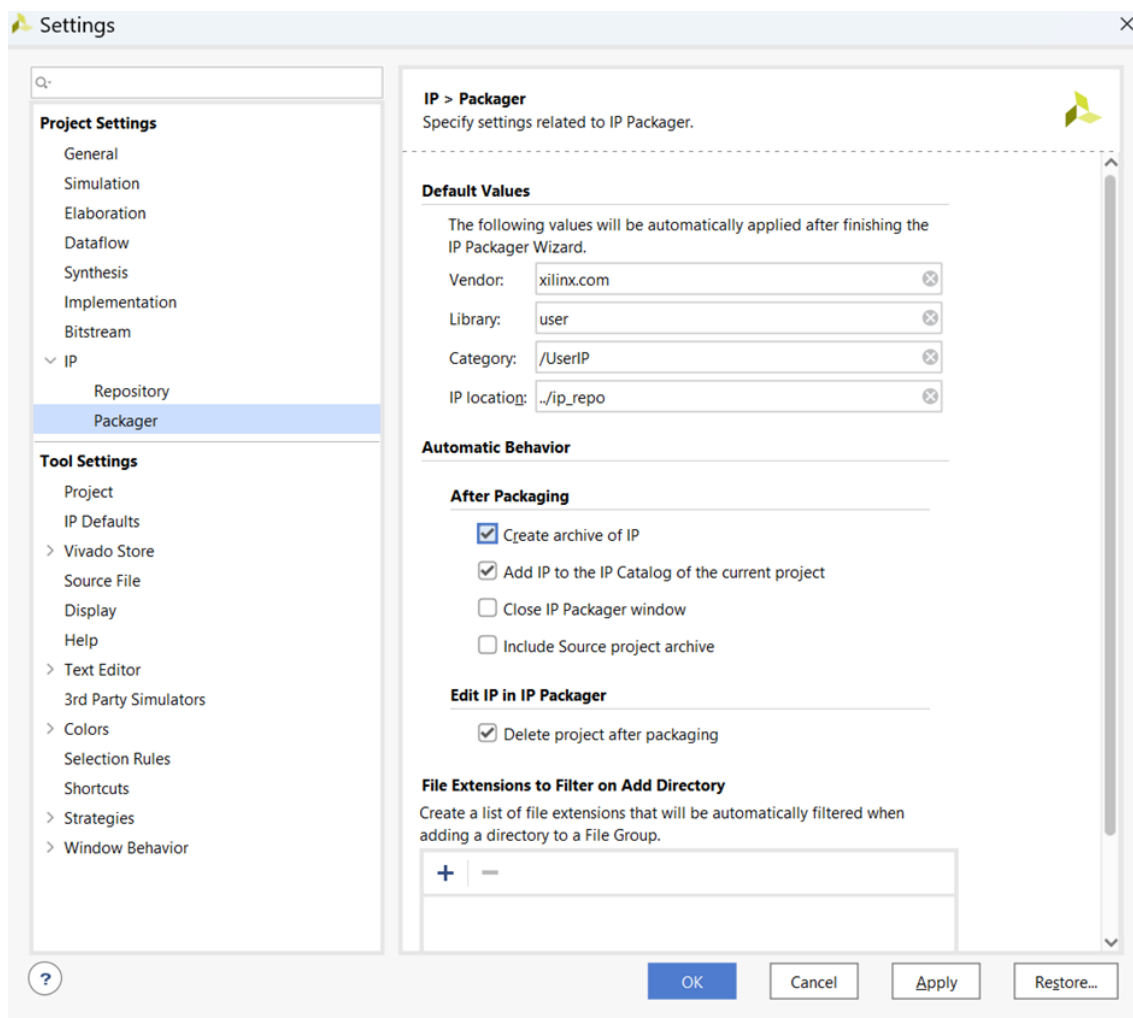
Hình 16 Tab Package IP

Chọn IP Customization Parameters, nhấn Merge changes from IP Customization Parameters Wizard. Sau đó vào IP Ports and Interfaces để kiểm tra đã xuất hiện cổng LEDs_out.



Hình 17 Tab IP Ports and Interfaces

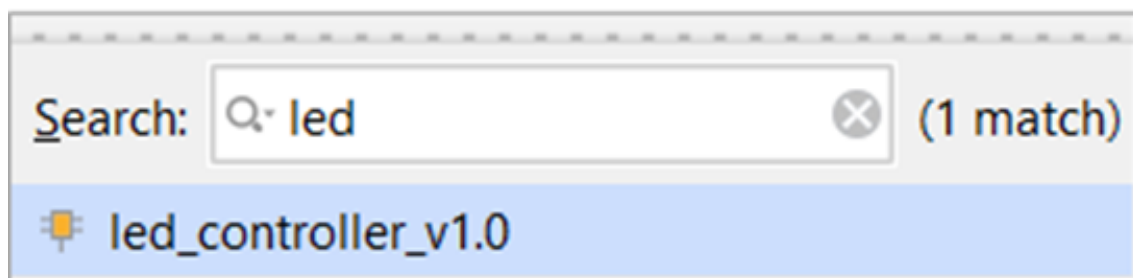
Cuối cùng chọn Review and Package, bật tùy chọn tạo archive, rồi nhấn Re-Package IP.



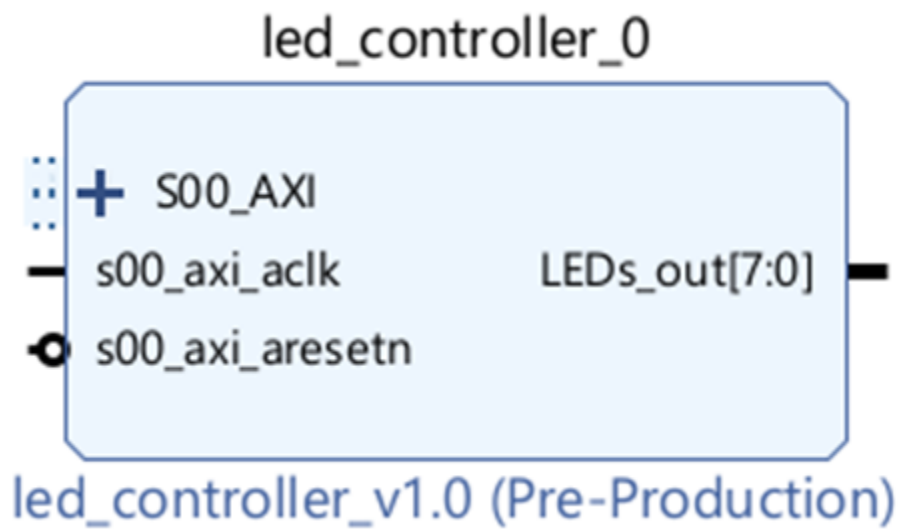
Hình 18 Bật Create archive of IP

Quay trở lại project chính, chọn mục Create Block Design trong tab Flow Navigator

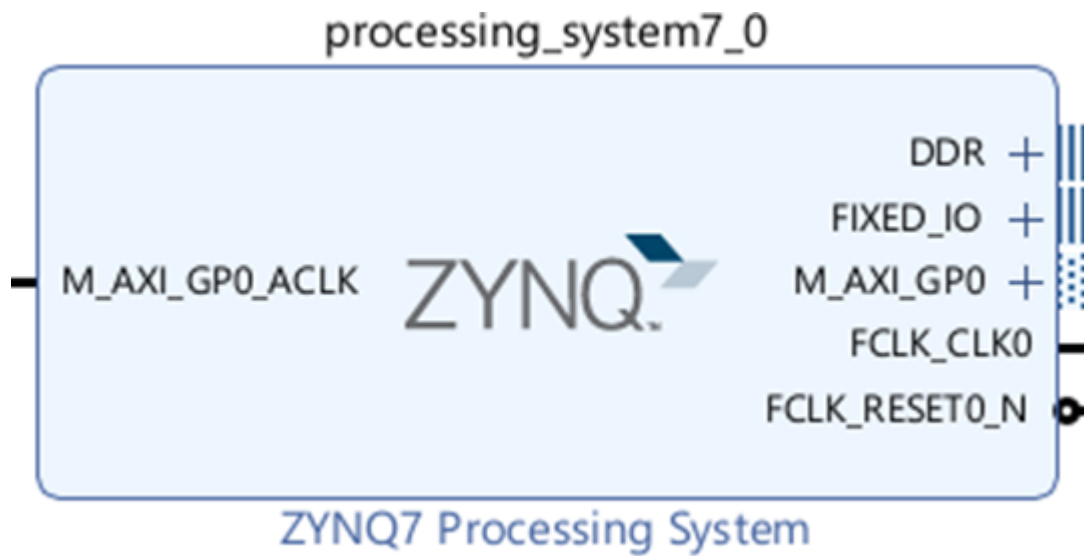
Nhấn chuột phải vào khoảng trống của tab Diagram, chọn led_controller_v1.0 và Zynq



Hình 19 Led_controller_v1.0

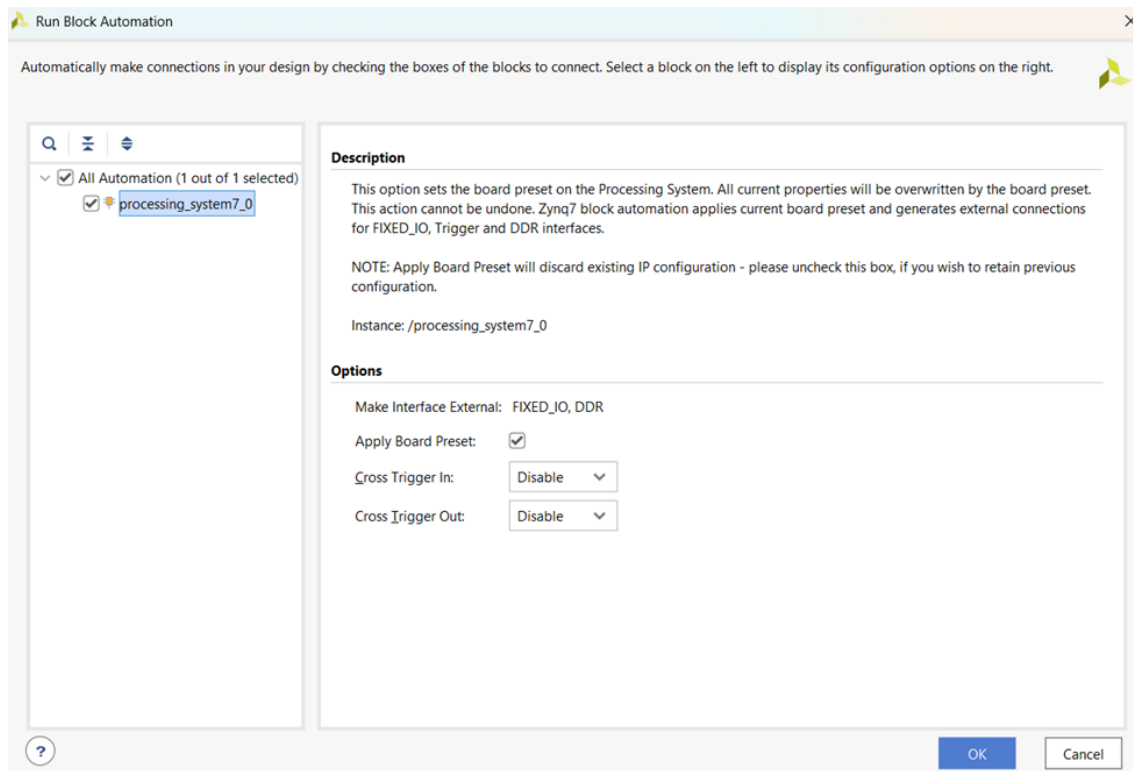


Hình 20 Khối led controller



Hình 21 Khối Zynq

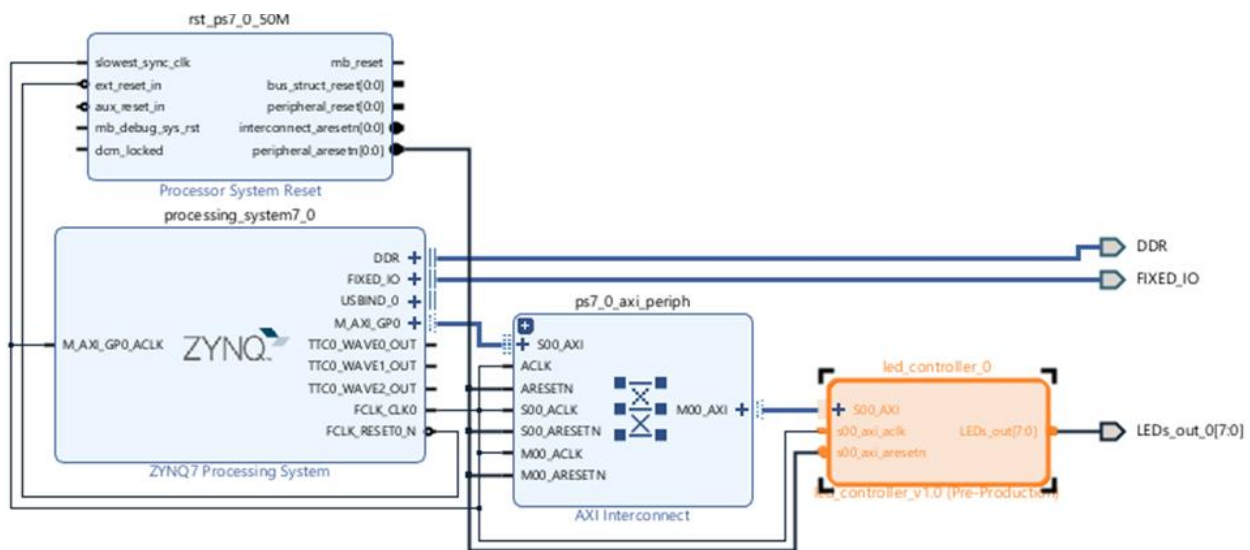
Chọn Run Block Automation



Hình 22 Cửa sổ Run Block Automation

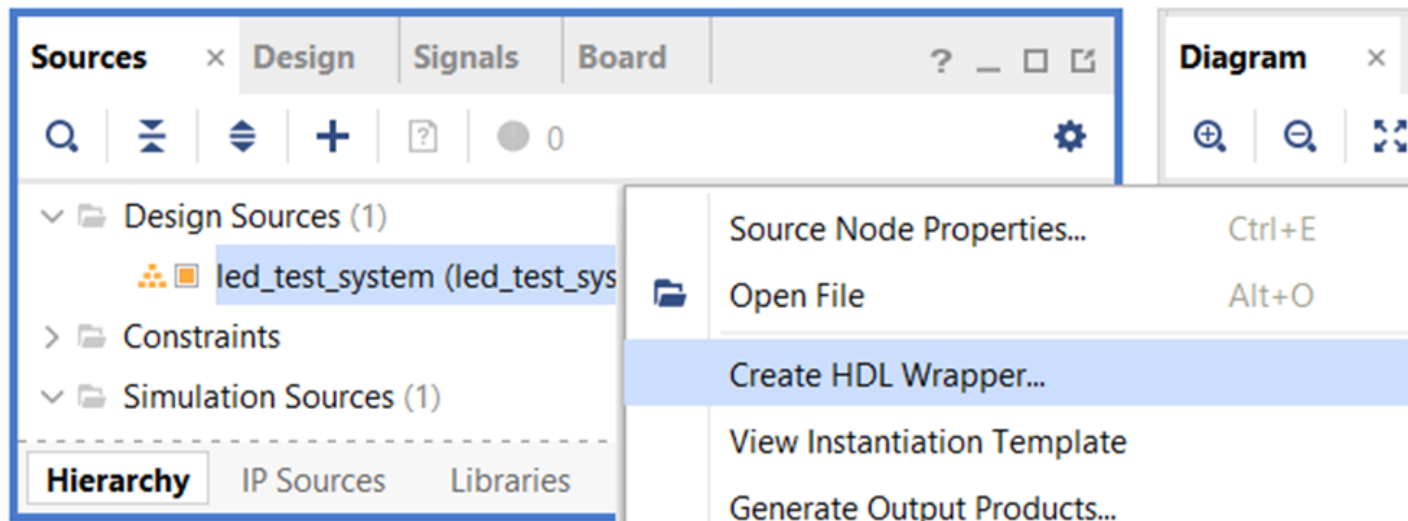
Tick chọn Apply Board Preset → OK

Chọn Run connection Automation



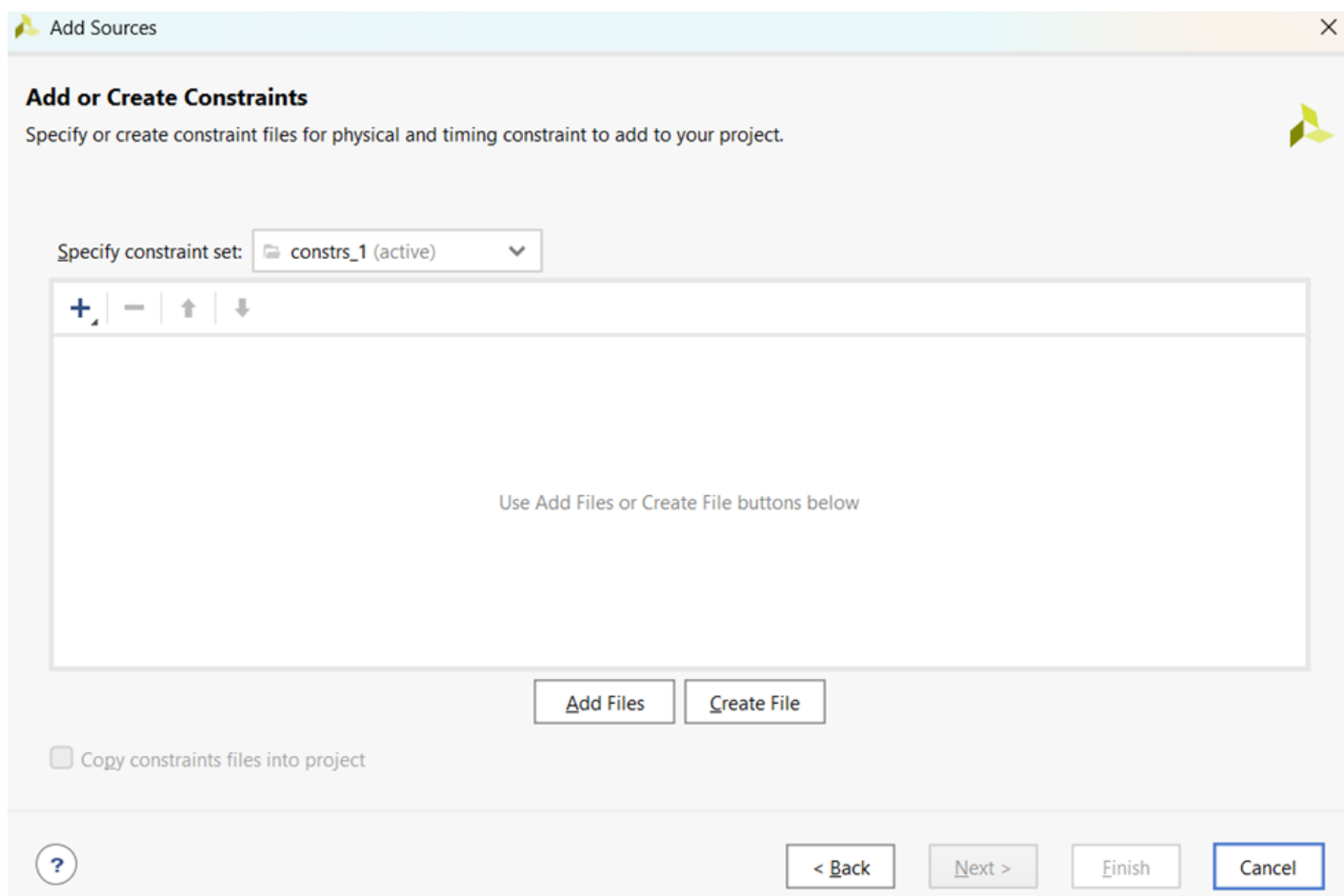
Hình 23 Sơ đồ khối

Kiểm tra xem mạch có lỗi không, chọn Tools → Validate Design



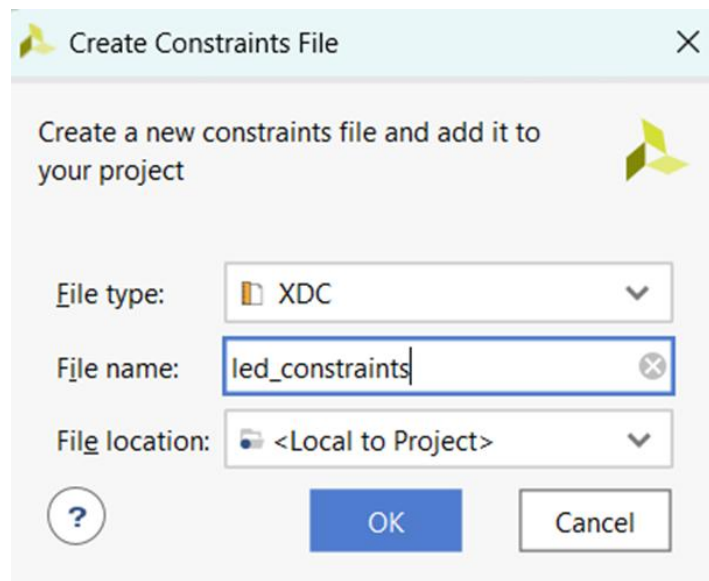
Hình 24 Create HDL Wrapper

Đóng gói lại, nhấn chuột phải vào led_test_system à Create HDL Wrapper



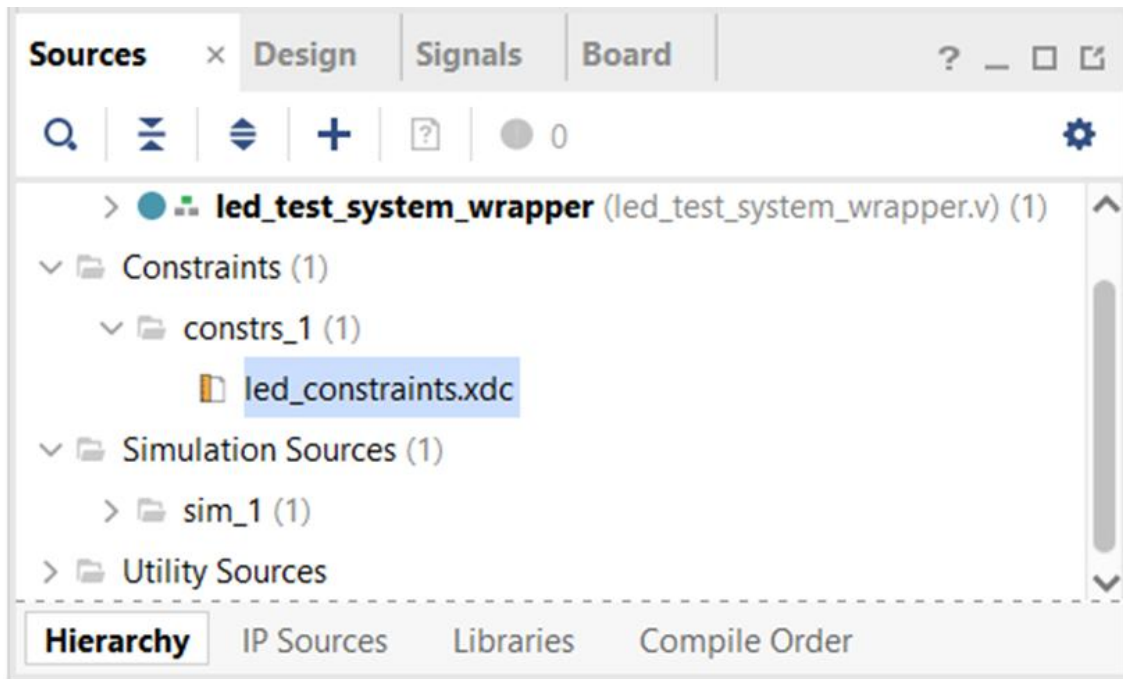
Hình 25 Cửa sổ Add Source

Chọn Add Source trong tab Flow Navigator → Add or create constraints → Next → Create file



Hình 26 Led constraints

Chọn loại XDC, tên led_constraints → Ok



Hình 27 Xuất hiện file trong tab Sources mục Constraints

Nhấn đúp chuột vào file led_constraints.xdc và nhập code bên dưới vào.


```

1  set_property PACKAGE_PIN T22 [get_ports {LEDs_out_0[0]}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[0]}]
3  set_property PACKAGE_PIN T21 [get_ports {LEDs_out_0[1]}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[1]}]
5  set_property PACKAGE_PIN U22 [get_ports {LEDs_out_0[2]}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[2]}]
7  set_property PACKAGE_PIN U21 [get_ports {LEDs_out_0[3]}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[3]}]
9  set_property PACKAGE_PIN V22 [get_ports {LEDs_out_0[4]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[4]}]
11 set_property PACKAGE_PIN W22 [get_ports {LEDs_out_0[5]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[5]}]
13 set_property PACKAGE_PIN U19 [get_ports {LEDs_out_0[6]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[6]}]
15 set_property PACKAGE_PIN U14 [get_ports {LEDs_out_0[7]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {LEDs_out_0[7]}]

```

Hình 28 Code file led_constraints.xdc

Packaging Steps

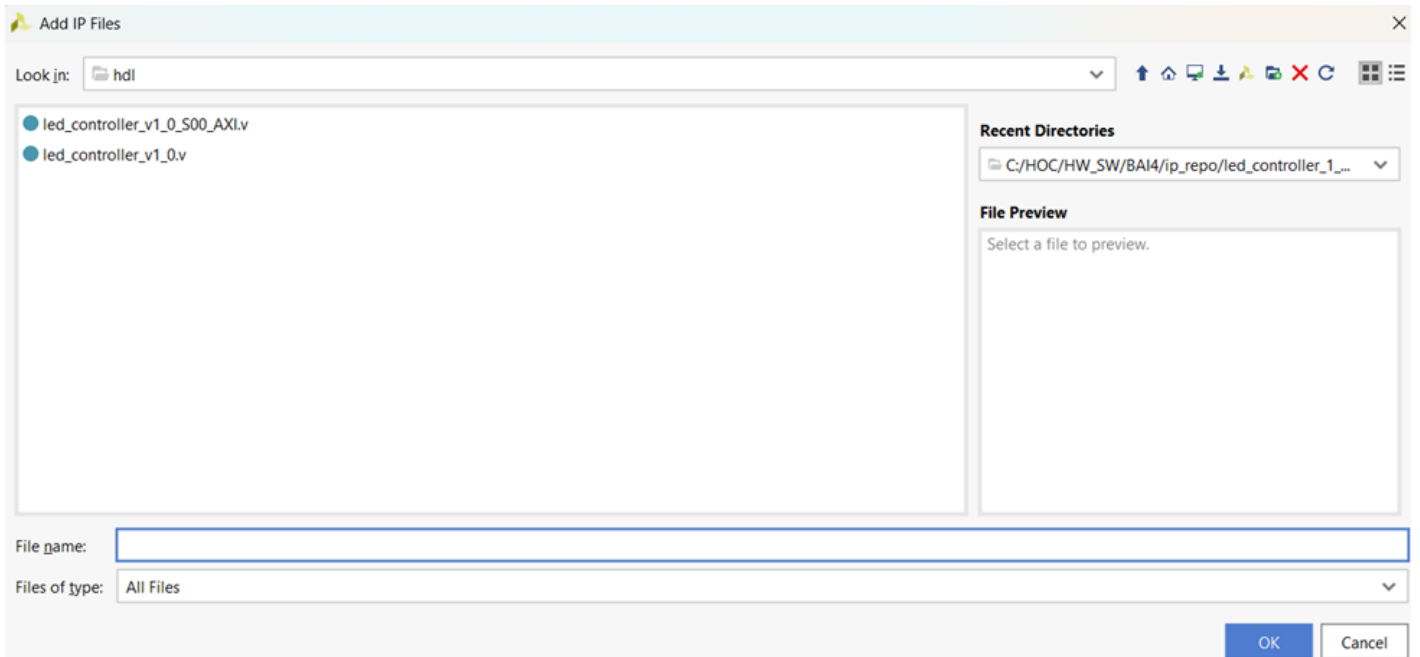
- ✓ Identification
- ✓ Compatibility
- ✓ File Groups
- ✓ Customization Parameters
- ✓ Ports and Interfaces
- ✓ Addressing and Memory
- ✓ Customization GUI
- ✓ Review and Package

File Groups

Name	Library Name	Type	Is Include	File Group Name	Model Name
Standard			☐		
Advanced			☐		
Verilog Synthesis (2)			☐		led_controller_v1_0
hdl/led_controller_v1		verilog	☐	xilinx_verilog	
hdl/led_controller_v1	xil_defaultlib	verilog	☐	xilinx_verilog	
Verilog Simulation (1)			☐		led_controller_v1_0
Software Driver (6)			☐		
UI Layout (1)			☐		
Block Diagram (1)			☐		

Hình 29 Tab File Groups trong Package IP

Trước khi Generate bitstream, kiểm tra tab Package IP- led_controller, mục File Group → Verilog Synthesis xem có đủ 2 file như trong hình chưa.



Hình 30 Add IP Files

Nếu thiếu thì nhấn chuột phải vào Verilog Synthesis chọn Add File, tới thư mục hdl, chọn loại file là All Files, và thêm file còn thiếu vào.

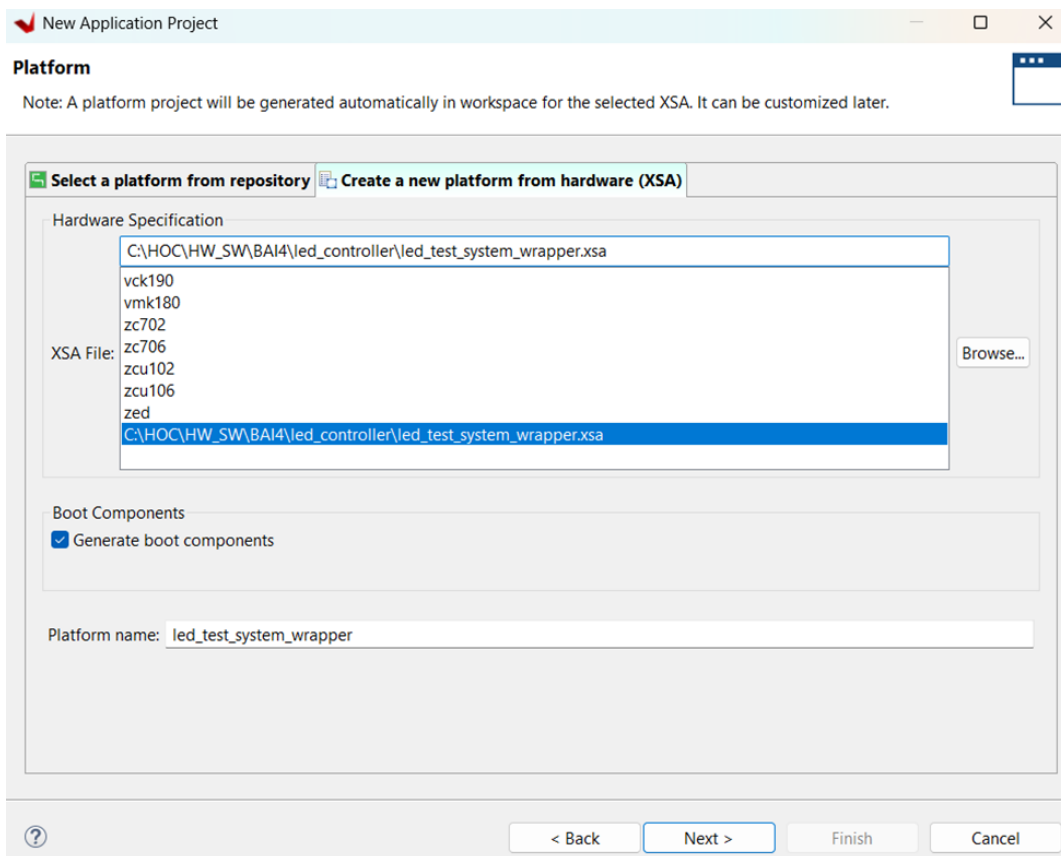
Quay lại project chính và chọn **Report IP Status -> Upgrade Selected.**

Sau đó Run Synthesis → Run implimentation → Generate Bitstream

Sau khi Generate Bitstream chạy xong thì chọn tools → Launch Vitis IDE

Chọn workspace mới

Create application project



Hình 31 Cửa sổ chọn platform cho New Application Project

Next -- > Next → Empty Application (C)

Phân tích hiệu suất:

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 12,656 ns	Worst Hold Slack (WHS): 0,058 ns	Worst Pulse Width Slack (WPWS): 9,020 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1493	Total Number of Endpoints: 1493	Total Number of Endpoints: 674

All user specified timing constraints are met.

Hình 32 Timing Summary

Setup Timing (WNS – Worst Negative Stack = 12.656ns): là khoảng thời gian dữ liệu phải ổn định trước khi có clock edge. WNS > 0 cho thấy dữ liệu đã đến sớm hơn yêu cầu. Thời gian dư tới 12.656ns cho thấy mạch đơn giản và tín hiệu truyền đi lập tức mà không có delay.

Hold Timong (WHS – Worst Hold Stack = 0.058 ns): là khoảng thời gian sau khi có edge clock mà dữ liệu vẫn giữ nguyên được trạng thái. Thời gian giữ nguyên là 0.058ns cho thấy dữ liệu thay đổi không quá nhanh, nhờ đó mà thanh ghi không đọc sai.

Pulse Width (WPWS - Worst Pulse Width Stack = 9.02ns): Là độ rộng xung, 9.02ns là độ rộng xung ổn định, đảm bảo clock không bị méo hay nhiễu tín hiệu.

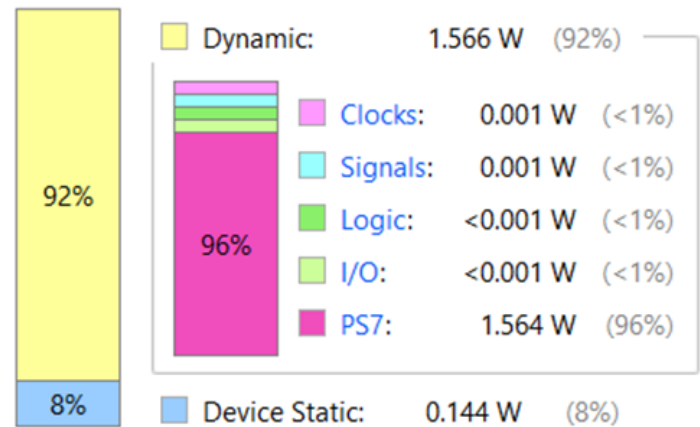
Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	1.71 W
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	44,7°C
Thermal Margin:	40,3°C (3,4 W)
Effective θ_{JA} :	11,5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



Hình 33 Summary Power

Tổng công suất trên chip (Total On-chip Power = 1.71W).

PS7: Khối xử lý và các ngoại vi chiếm đa phần công suất động,

Junction Temperature: 44.7°C là một nhiệt độ khá mát đối với chip xử lý.

Effective: 11.5°C/W nghĩa là với mỗi công suất tỏa ra thì lõi chip sẽ nóng thêm 11.5°C, theo công thức tính như sau:

Với: T_A : Nhiệt độ phòng (thường là 25°C).

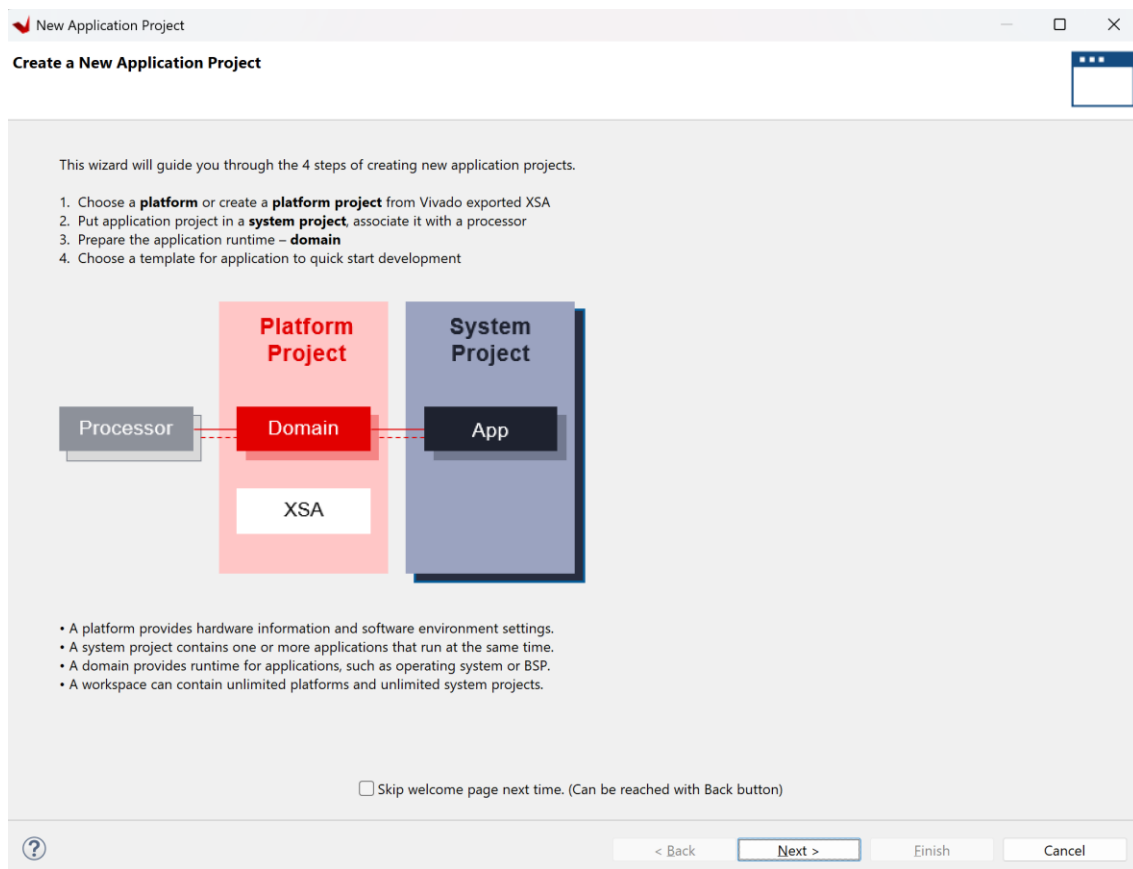
P_{total} : Công suất tổng.

θ_{JA} : Hệ số Effective.

Thermal Margin: 40.3°C (3.4W). Cho biết chip còn cách ngưỡng nhiệt độ hoạt động là 40.3°C, tức là có thể thêm các khối thiết kế tiêu thụ tối đa thêm 3.4W trước khi phải gắn quạt tản nhiệt.

Tạo project Vitis đến import file code C

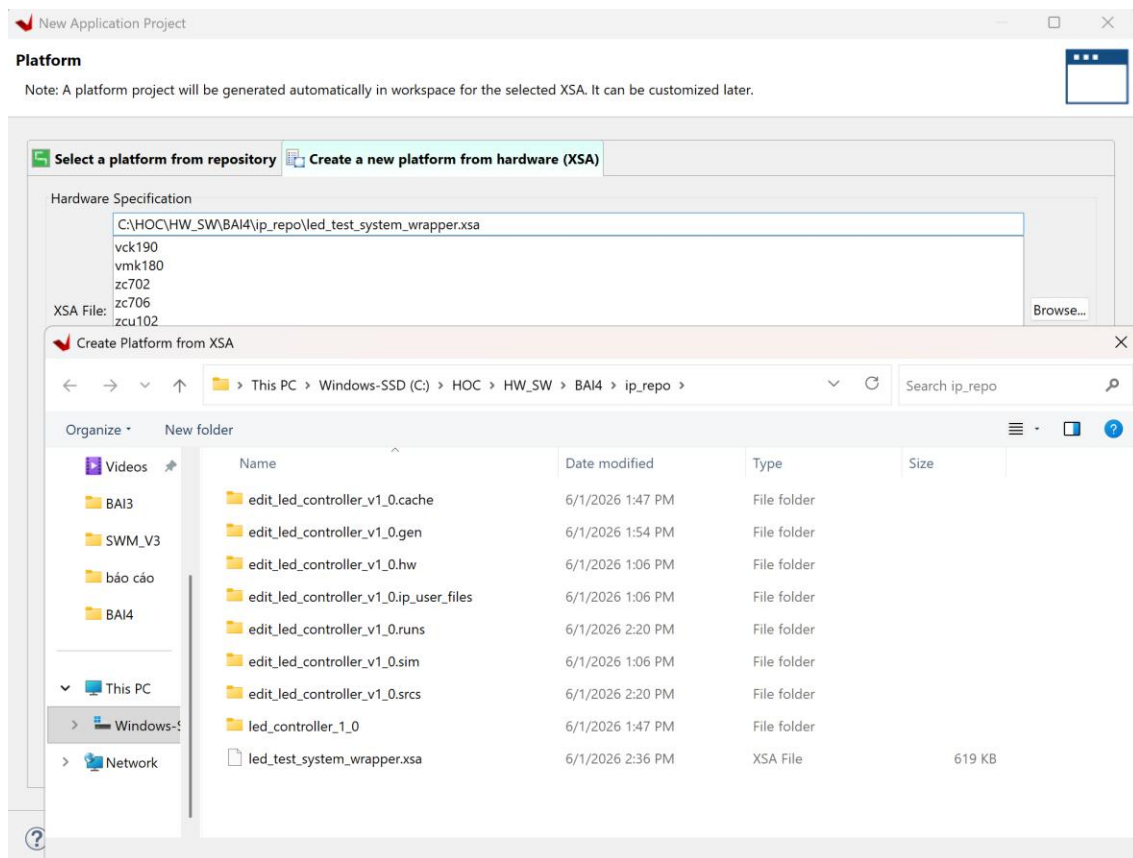
Sau khi hoàn tất quá trình thiết kế phần cứng, tạo HDL Wrapper, Generate Bitstream và Export Hardware từ Vivado, bước tiếp theo là sử dụng Vitis IDE để xây dựng chương trình phần mềm chạy trên bộ xử lý ARM của Zynq. Chương trình này có nhiệm vụ ghi dữ liệu vào IP tùy chỉnh thông qua giao tiếp AXI-Lite, từ đó điều khiển trạng thái các LED trên board.



Hình 34 Tạo mới ở Vitis IDE

Chọn next,

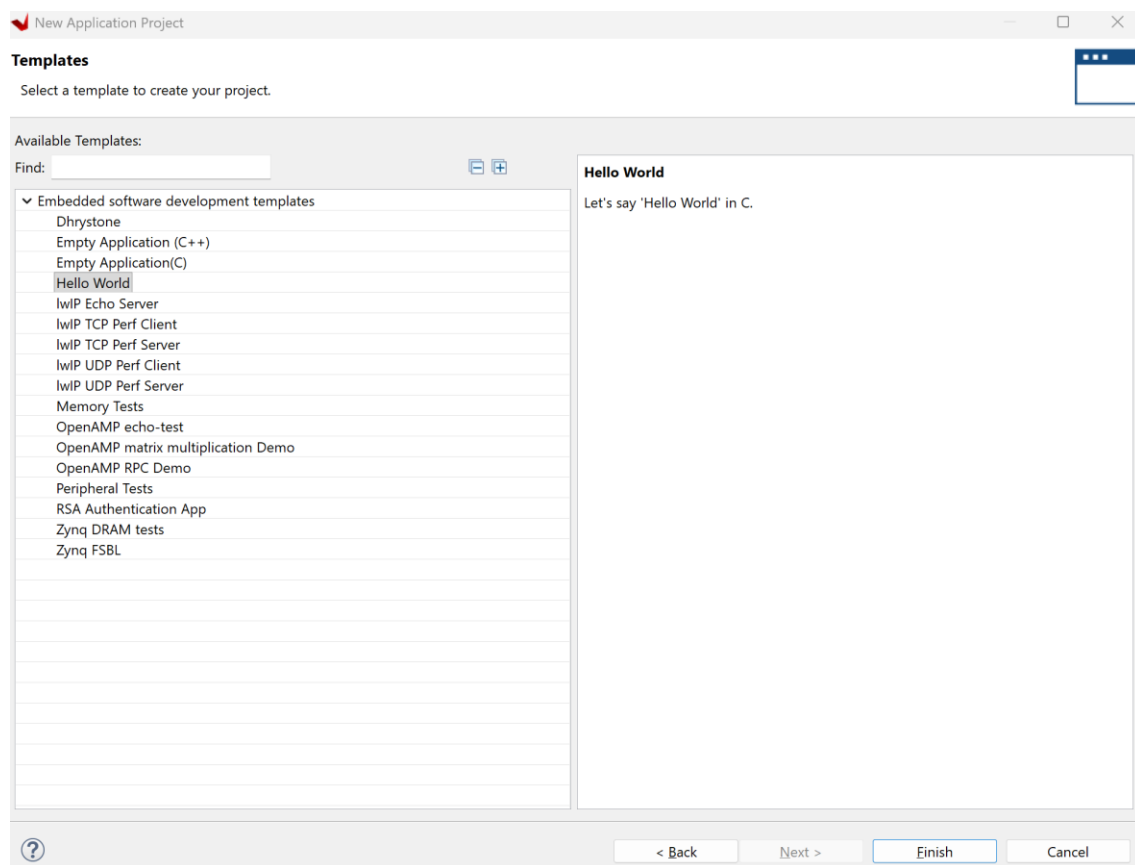
Đây là bước khởi tạo project phần mềm dùng để viết và biên dịch chương trình chạy trên Processing System. Vitis tổ chức project theo mô hình gồm Platform Project, System Project và Application Project. Trong đó, Platform Project chứa thông tin phần cứng được export từ Vivado, còn Application Project chứa chương trình C/C++ do người dùng xây dựng.



Hình 35 Chọn Platform

Chọn file .xsa , nhấn open rồi next.

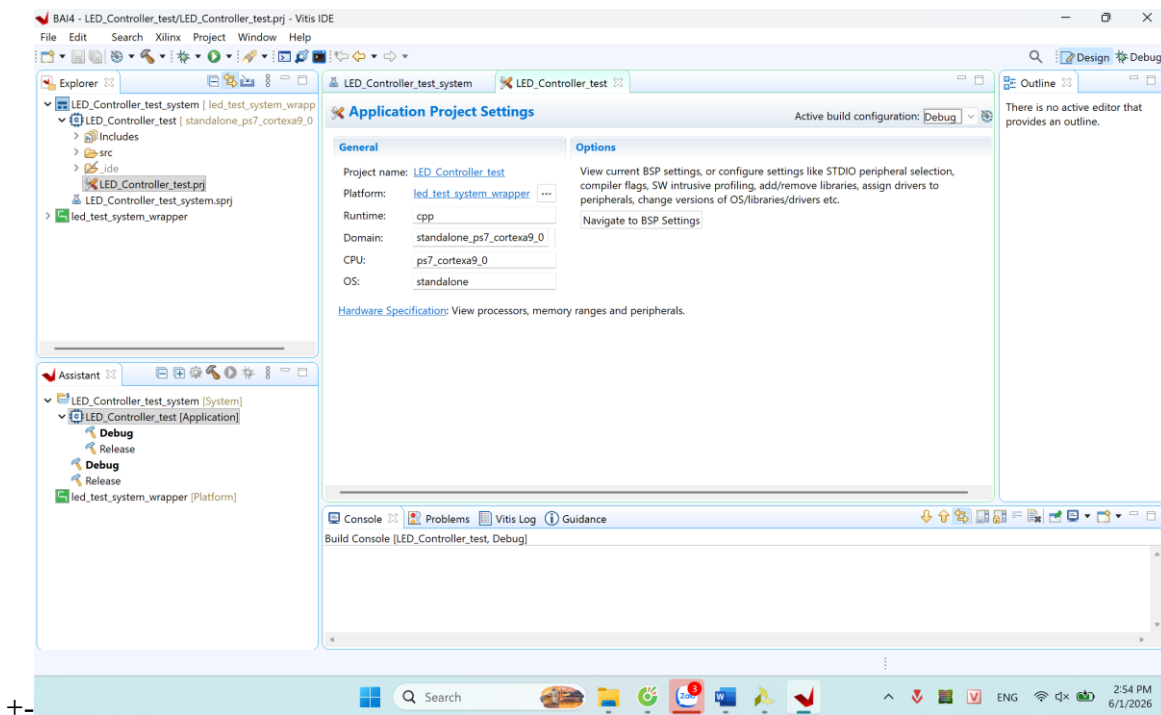
File .xsa chứa toàn bộ thông tin của hệ thống phần cứng, bao gồm Zynq Processing System, AXI Interconnect, custom IP led_controller và bitstream.



Hình 37 Chọn Templates

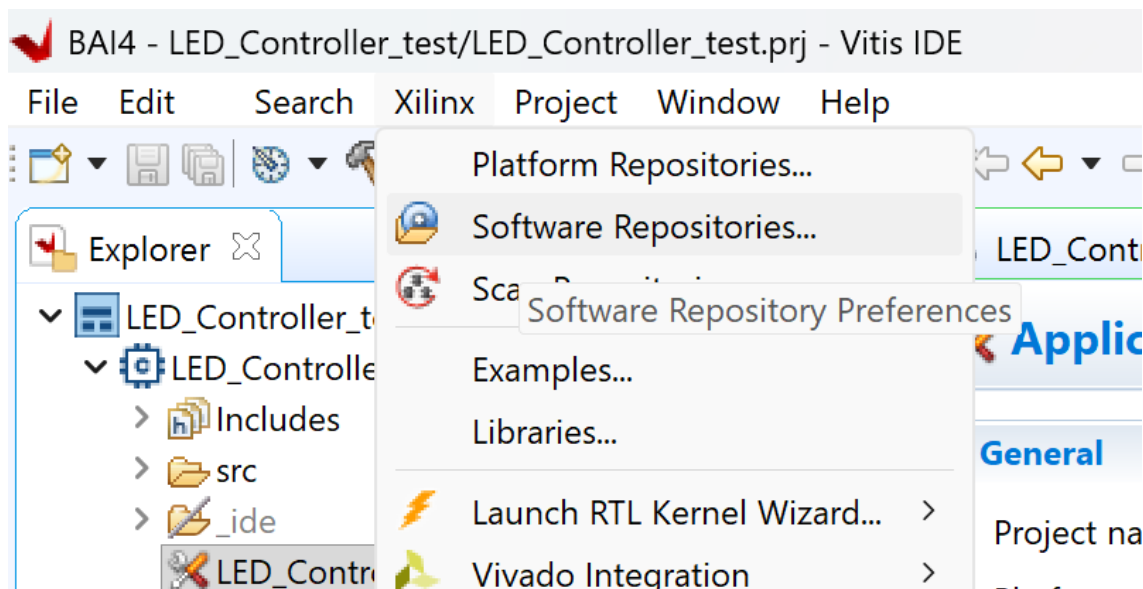
Ở phần templates chọn Empty Application(C) rồi Finish.

Sau khi xong được trang chủ như này.



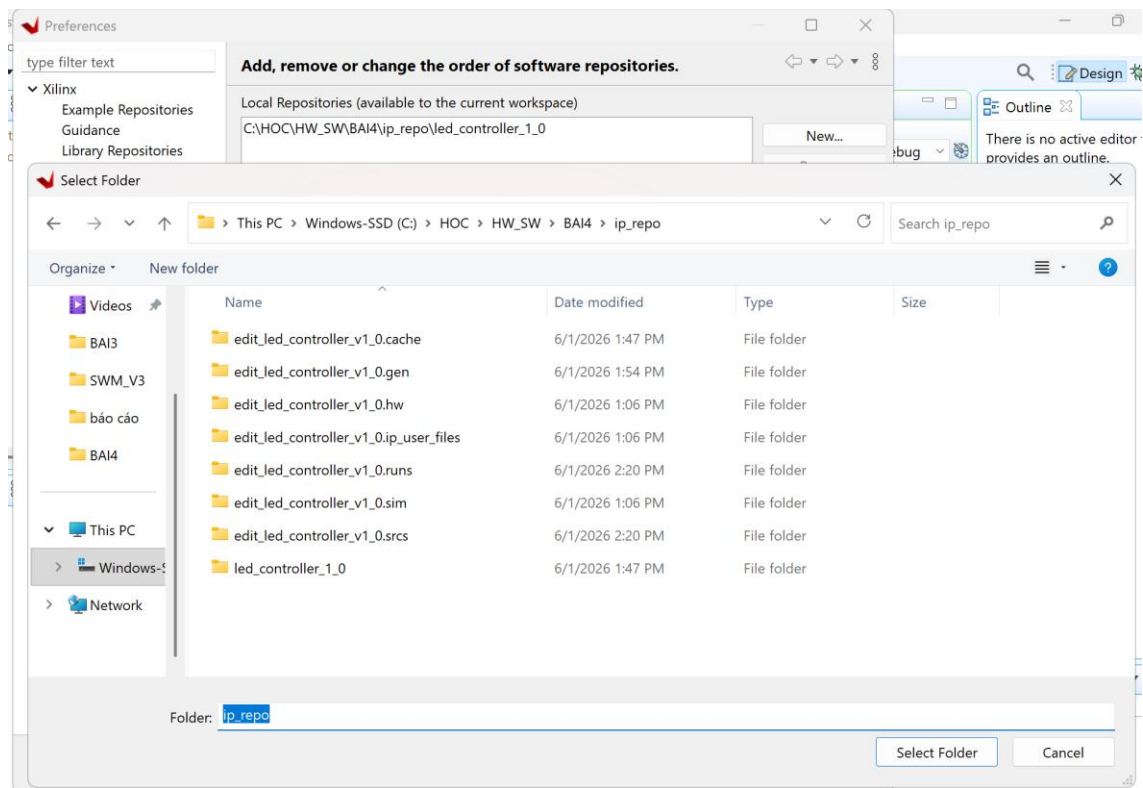
Hình 38 Màn hình sau khi tạo

Sau khi tạo project phần mềm, cần bổ sung repository chứa driver hoặc các file hỗ trợ cho IP tùy chỉnh. Trong Vitis IDE, thao tác này được thực hiện thông qua menu Xilinx > Software Repositories. Đây là bước tương ứng với thao tác thêm repository trong SDK cũ, giúp Vitis nhận diện các thành phần phần mềm liên quan đến IP do người dùng tự tạo.

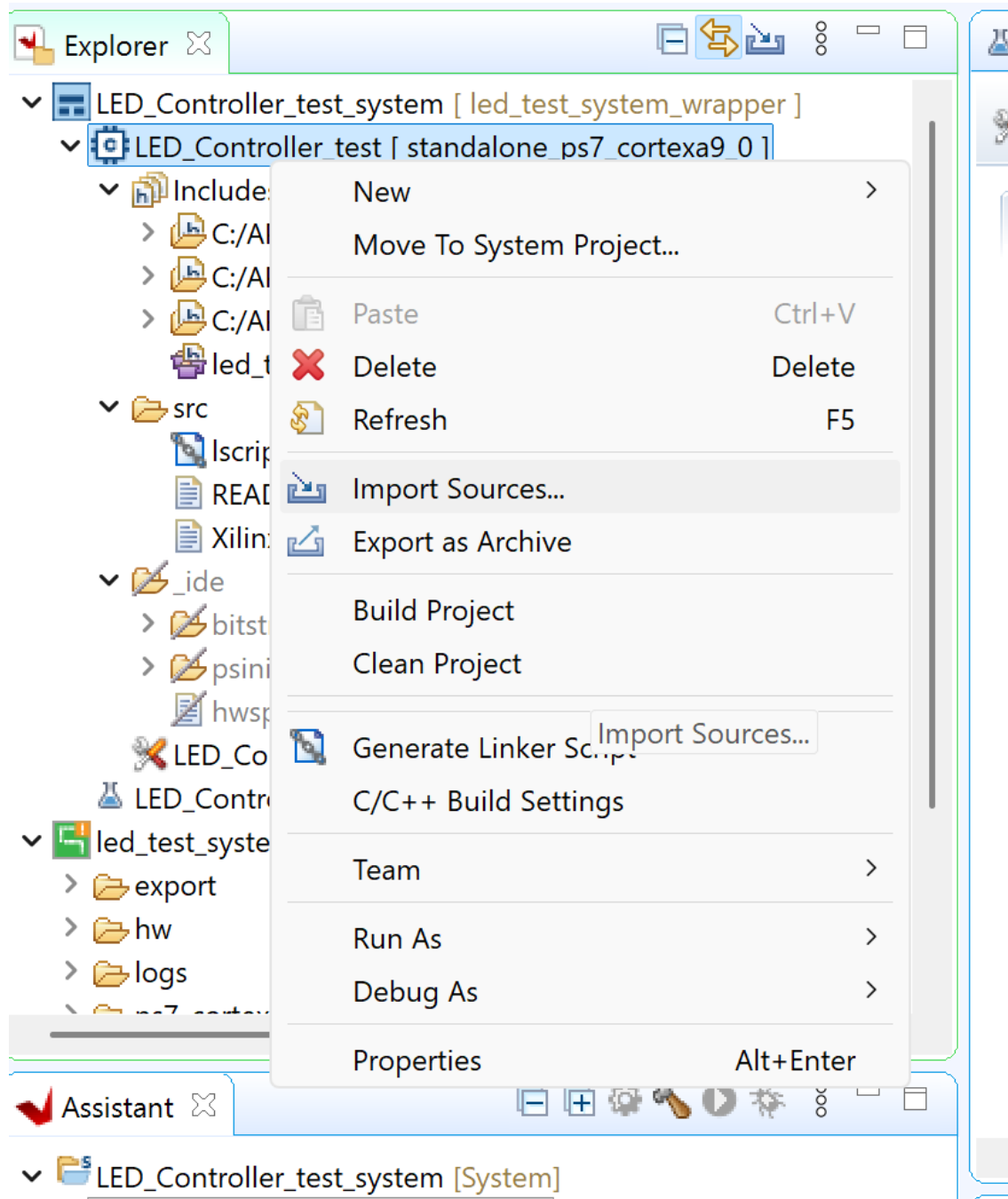


Hình 39 Software Repositories

Chức năng này cho phép thêm đường dẫn đến thư mục chứa driver hoặc thư viện phần mềm của custom IP. Đối với bài điều khiển LED, repository được thêm nhằm hỗ trợ quá trình biên dịch và sử dụng IP led_controller trong project phần mềm.

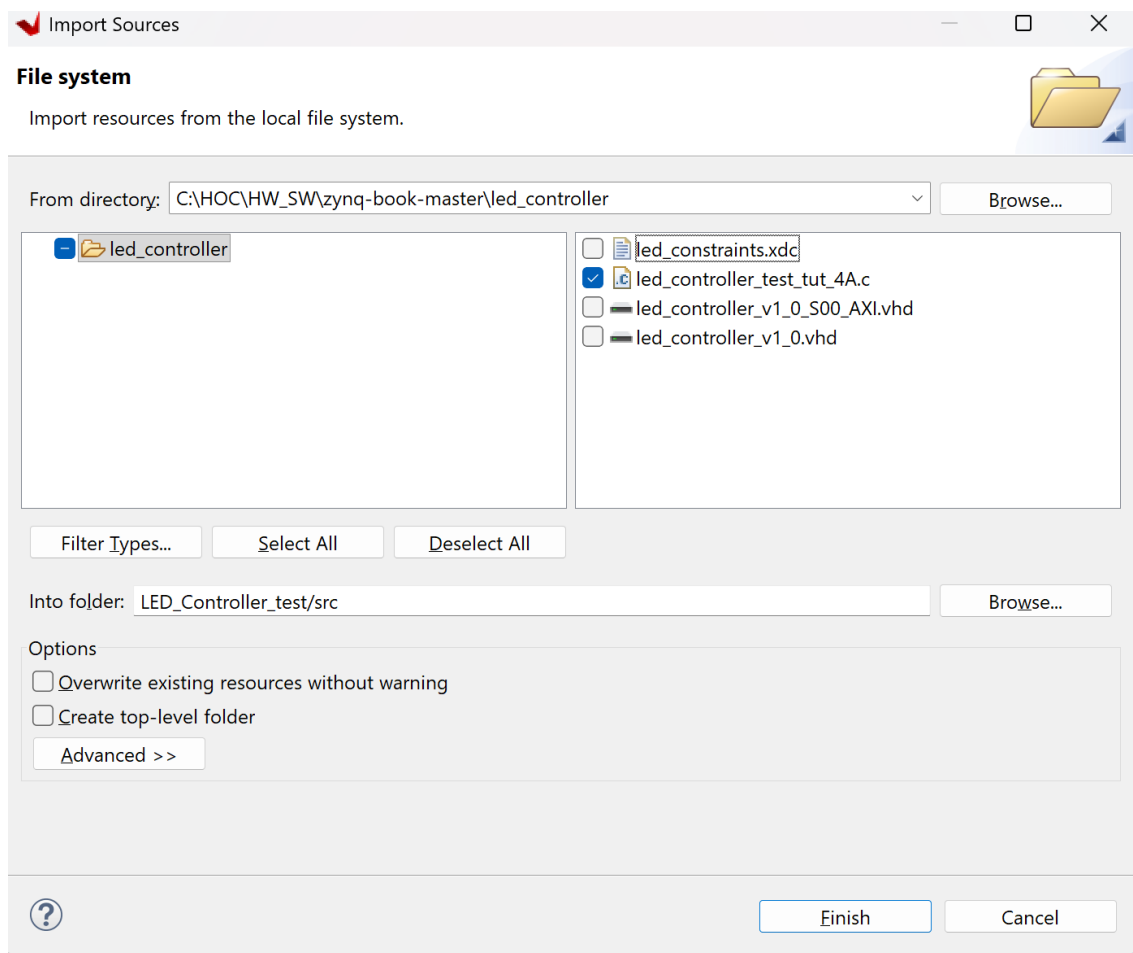


Hình 40 Chọn ip-repo



Hình 41 Import sources

File chương trình C được thêm vào thư mục src của project LED_Controller_test. Đây là file chứa chương trình điều khiển LED, có nhiệm vụ ghi các giá trị dữ liệu xuống thanh ghi của IP thông qua AXI-Lite. Khi chương trình chạy trên PS, dữ liệu được ghi vào thanh ghi slv_reg0, sau đó xuất ra cổng LEDs_out[7:0] để điều khiển các LED.

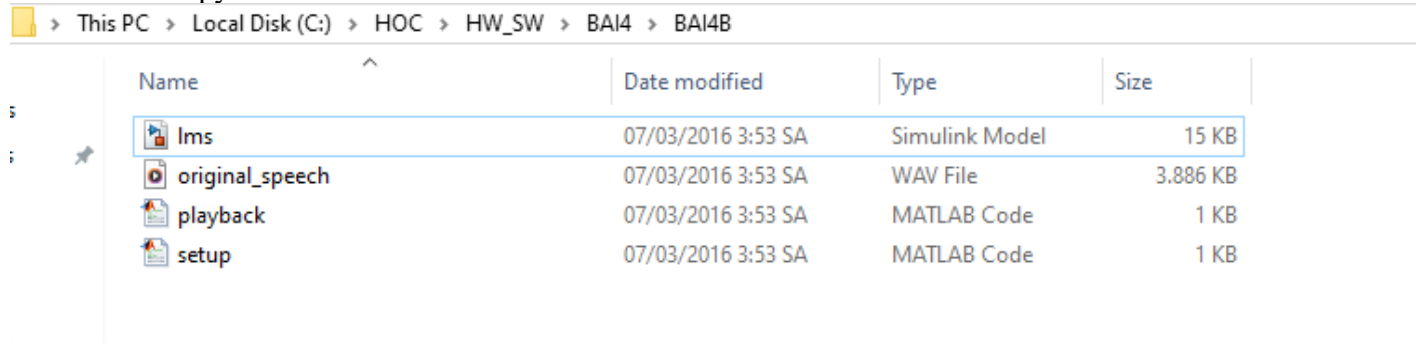


Hình 42 led_constraints

Trong bước này, file led_controller_test_tut_4A.c được chọn để đưa vào project phần mềm. Các file HDL hoặc constraint không cần import vào project C vì chúng đã được sử dụng trong Vivado ở giai đoạn thiết kế phần cứng. Sau khi import file C, project có thể được build và chạy trên phần cứng để kiểm tra khả năng điều khiển LED từ phần mềm.

Exercise 4B: Creating IP in MathWorks HDL Coder

B1: Copy các file code mẫu về folder cho 4B

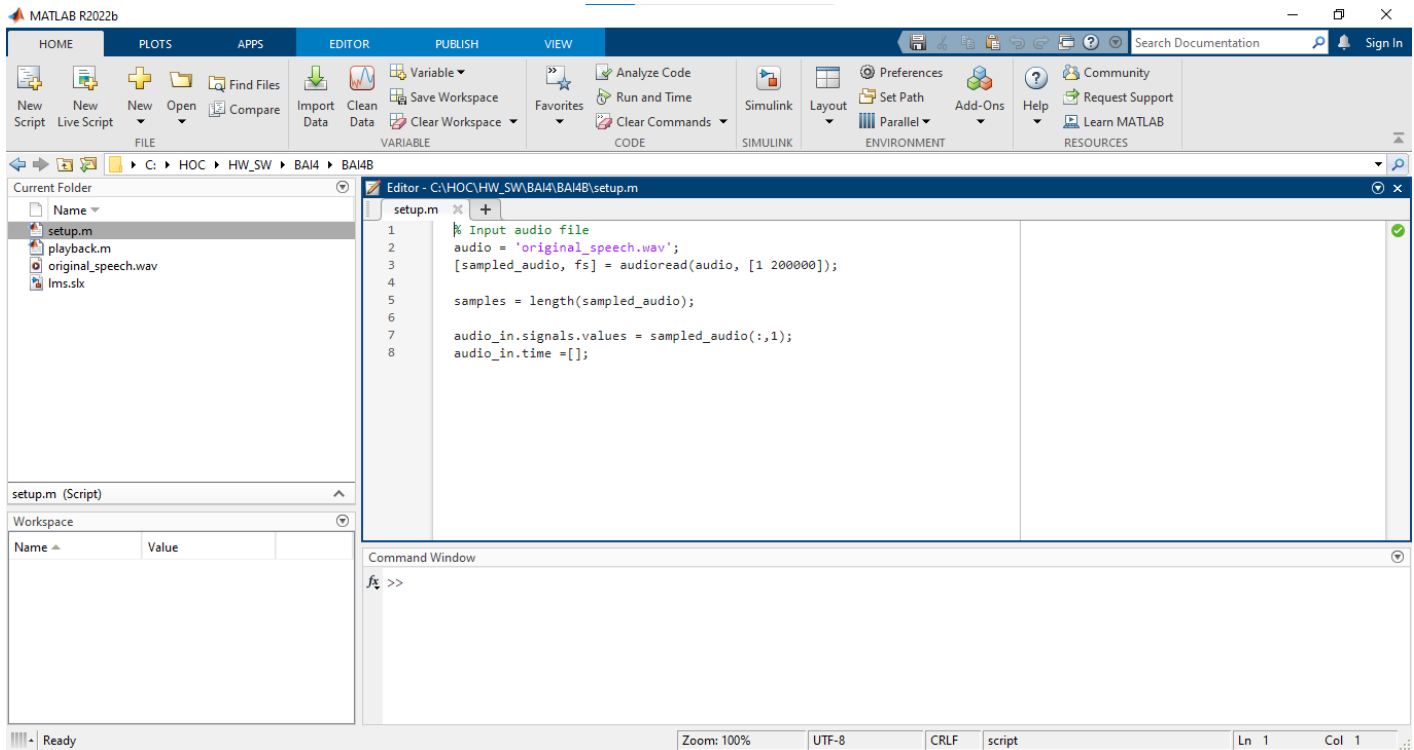


Hình 43 Các file cần thiết

- **original_speech.wav**: là 1 đoạn âm thanh tiếng nói
- **setup.m**: Thực hiện các lệnh thiết lập để nhập (import) các mẫu âm thanh vào không gian làm việc (workspace) của MatLab và cài đặt tốc độ lấy mẫu (sample rate) của hệ thống cho phù hợp
- **lms.slx** — Một mô hình Simulink thực hiện quy trình khử nhiễu LMS.

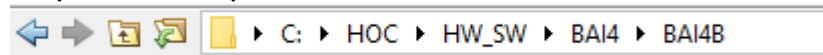
- **playback.m** — Có thể được sử dụng để kiểm chứng quá trình lọc LMS thông qua việc phát lại âm thanh ở các giai đoạn khác nhau

Bước 2: Mở MatLab



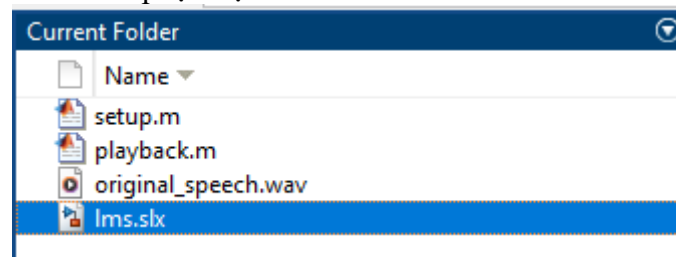
Hình 44 Cửa sổ mặc định của MatLab

Bước 3: Trở tới thư mục đã chuẩn bị



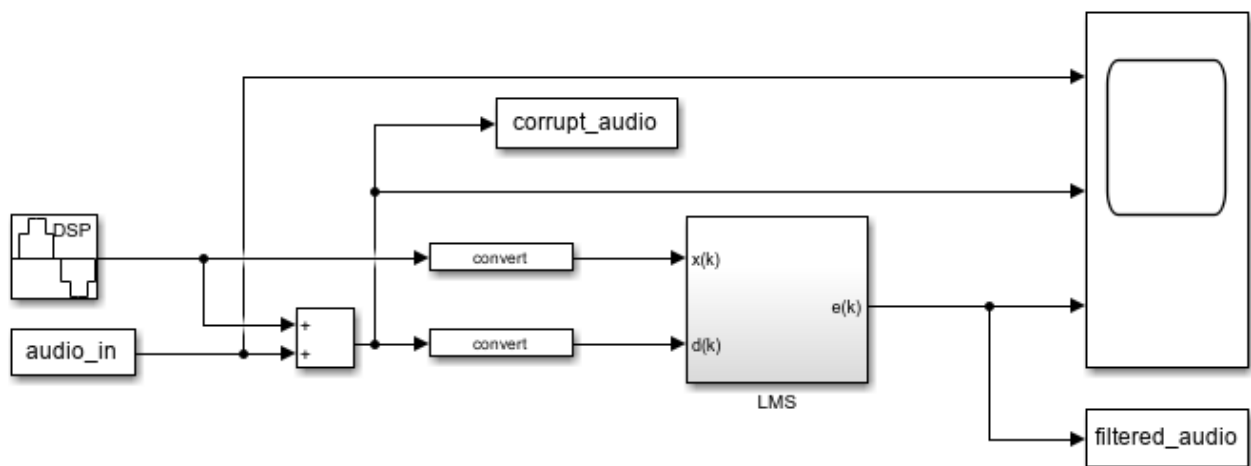
Hình 45 Đường dẫn

Bước 4: Chạy file lms.slx. Nhấn đúp tại mục Current Folder



Hình 46 Cửa sổ Current Folder

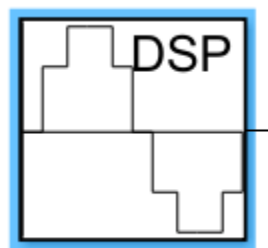
Khi chạy file 1 cửa sổ simulink sẽ hiện lên:



Hình 47 LMS model Simulink

Các khối Input:

Khối 1: SinWave

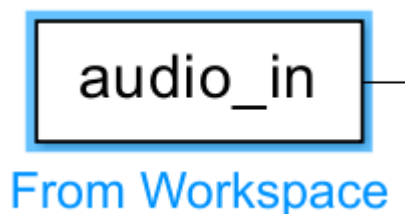


Sine Wave

Hình 48 Khối tạo nhiễu

Mục tiêu khối này là tạo các nhiễu âm sắc dưới dạng sóng sin để mô phỏng âm thanh thu thực tế.

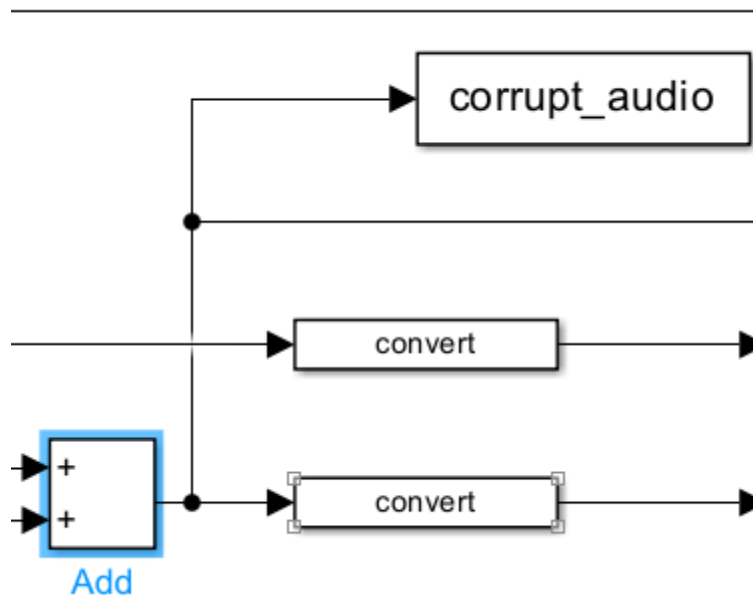
Khối 2: audio_in



Hình 49 Khối vào

Khối này có nhiệm vụ trích xuất dữ liệu tín hiệu giọng nói sạch (file original_speech.wav) đã được nạp sẵn vào không gian làm việc của MATLAB để đưa vào hệ thống mô phỏng.

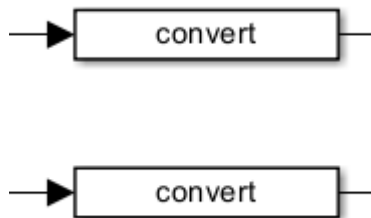
Các khối tạo nhiễu:



Khối tạo nhiễu Add và tín hiệu nhiễu Corrupt_audio

Khối Add trộn tín hiệu bằng cách cộng trực tiếp tín hiệu âm thanh sạch với nhiễu sóng hình sin, từ đó tạo ra một tín hiệu âm thanh bị hỏng/nhiều

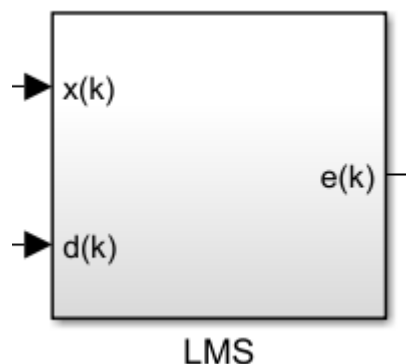
Các khối Chuyển đổi dữ liệu



Hình 50 Khối chuyển dữ liệu về fix-pointed

Tín hiệu mô phỏng thông thường trong Simulink nằm ở dạng số thực dấu phẩy động (floating-point). Tuy nhiên, mã phần cứng (HDL) bằng công cụ HDL Coder cho mạch phần cứng FPGA yêu cầu xử lý tín hiệu ở định dạng **số thực dấu phẩy tĩnh (fixed-point)**, hai khối này được dùng để ép kiểu tín hiệu nhiễu và tín hiệu âm thanh lỗi sang định dạng fixed-point tương thích trước khi đưa vào bộ xử lý.

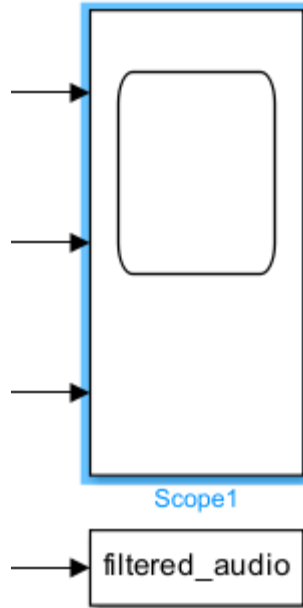
Lõi xử lý trung tâm (LMS Subsystem):



Hình 51 LMS

- Khối **LMS** nhận hai đường tín hiệu đầu vào:
 - + **$x(k)$ (Reference Input)**: Nhận tín hiệu nhiễu gốc từ khối Sine Wave. Thuật toán sẽ dùng tín hiệu này làm mẫu tham chiếu để phân tích đặc tính của nhiễu.
 - + **$d(k)$ (Desired Input)**: Nhận tín hiệu âm thanh đã bị trộn lẫn nhiễu.
 - + **$e(k)$ (Error Output)**: Thuật toán LMS sẽ liên tục điều chỉnh các hệ số nội bộ để trừ đi thành phần nhiễu có trong $d(k)$. Do đó, tín hiệu sai số $e(k)$ sinh ra ở đầu ra chính là thành quả: phần âm thanh đã được lọc sạch nhiễu.

Các khối Output:

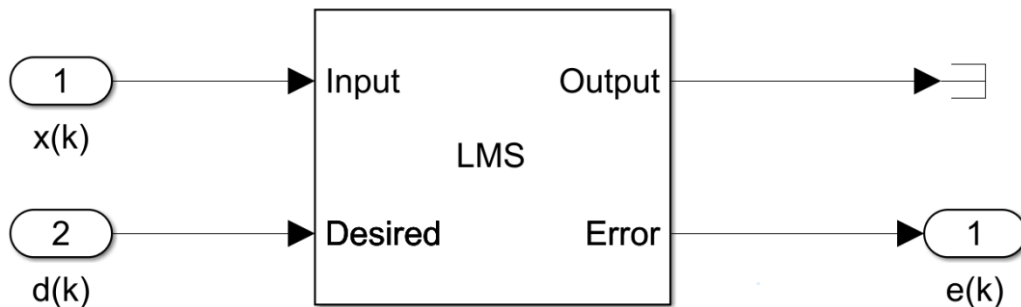


Hình 52 Khối quan sát Scope và âm thanh đã lọc

Scope: Đóng vai trò như một máy hiện sóng đa kênh. Khối này thu thập tín hiệu đầu vào (âm thanh lỗi, sóng nhiễu) và tín hiệu đầu ra $e(k)$ để bạn có thể quan sát, so sánh sự khác biệt dạng sóng trực quan trên màn hình.

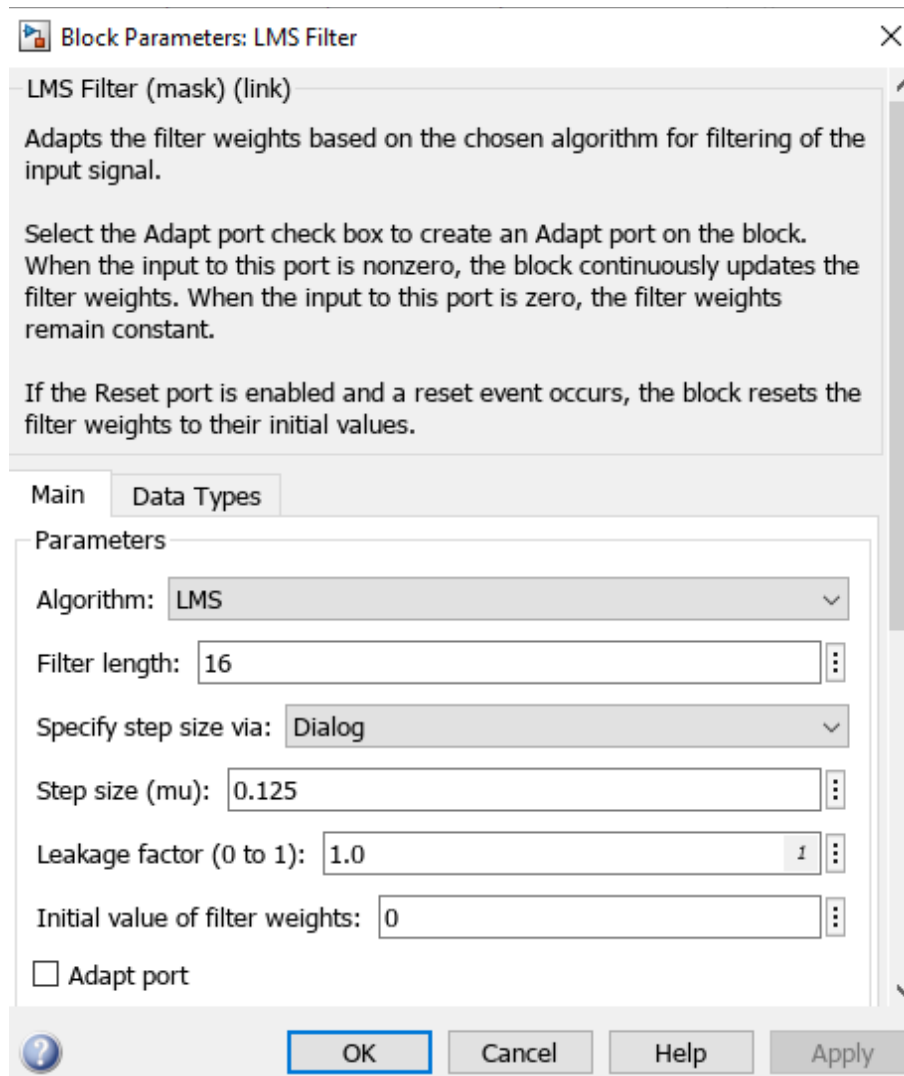
filtered_audio: Khối này có chức năng đẩy các luồng dữ liệu kết quả (âm thanh chưa lọc và đã lọc) ngược trở lại lưu trữ ở môi trường MATLAB Workspace. Nhờ đó có thể chạy file script để nghe thử trực tiếp thành quả lọc âm thanh bằng loa của máy tính.

Bước 5: Click đúp chuột vào khối LMS để quan sát cấu trúc nó



Hình 53 Cấu trúc khối LMS

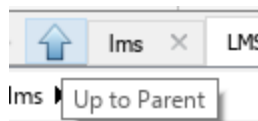
Bước 6: Tiếp tục click đúp vào khối LMS để xem bảng thông số



Hình 54 Các thông số của bộ lọc

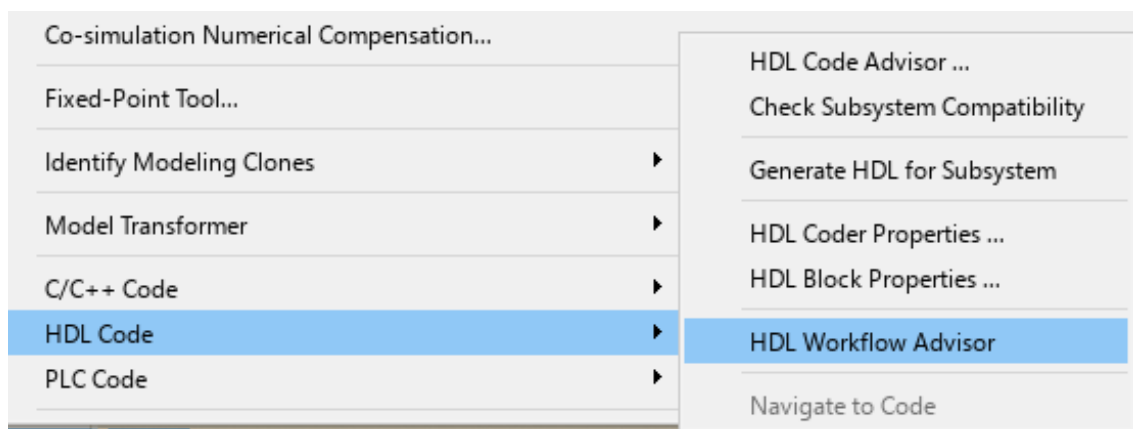
Bộ lọc này đang được cấu hình với 16 hệ số lọc (filter coefficients) và bước nhảy (step size) là 0.125.

Bước 7: Nhấn vào Up to parent để trở về file model đầu tiên

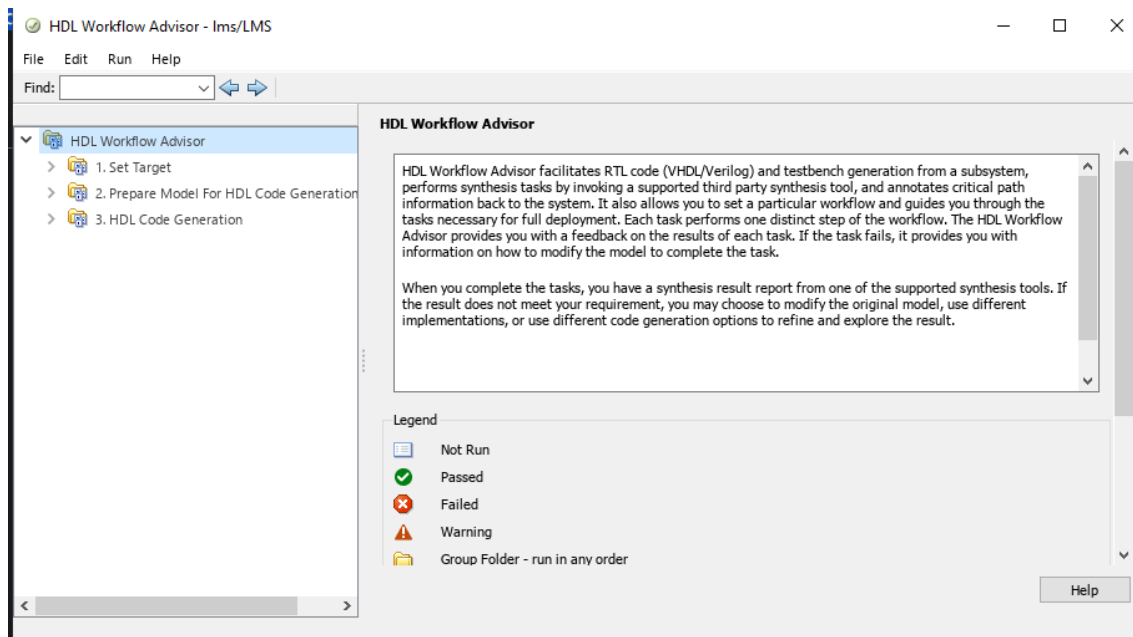


Hình 55 Up to Parent

Bước 8: Để tạo IP cho khối LMS ta nhấn phải chuột vào khối LMS > chọn HDL Code> chọn HDL Workflow.

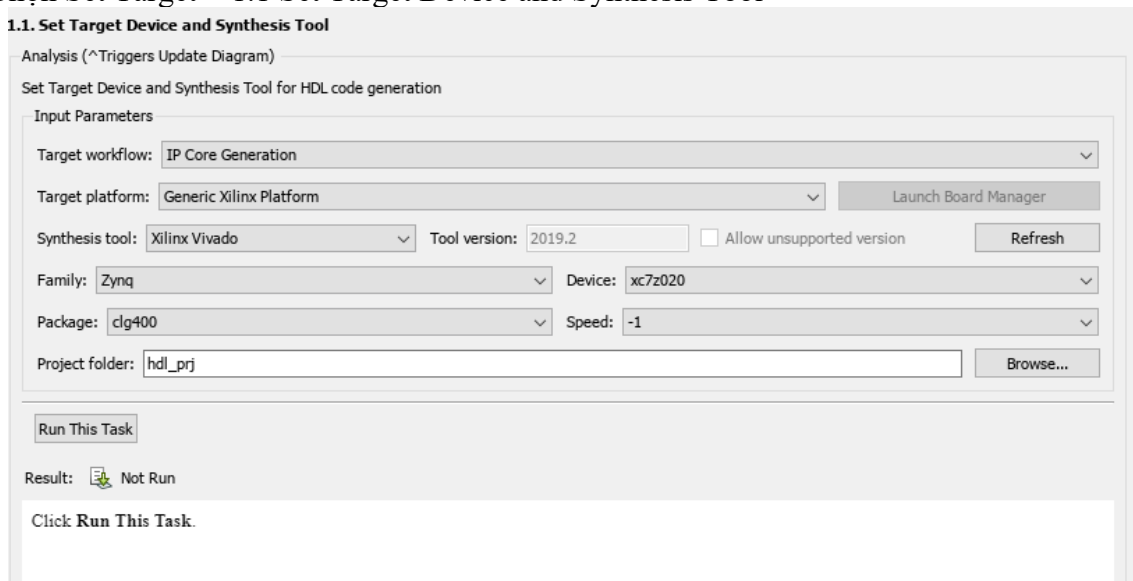


Hình 56 Bảng chọn khi phải chuột



Hình 57 HDL Workflow Advisor

Bước 9: Chọn Set Target > 1.1 Set Target Device and Synthesis Tool



Hình 58 Set up devices

Nếu bị lỗi không tìm được platform và synthesis tool>Tắt cửa sổ HDL Workflow Advisor> Quay về giao diện chính của MatLab> Command windows nhập: **hdlsetuptoolpath('ToolName','Xilinx Vivado','ToolPath','D:\vitis\Vivado\2019.2\bin\vivado.bat')**
Đường dẫn: **D:\vitis\Vivado\2019.2\bin\vivado.bat** sẽ thay đổi tùy vào vị trí lưu của Vivado

```
>> hdlsetuptoolpath('ToolName', 'Xilinx Vivado', 'ToolPath', 'D:\vitis\Vivado\2019.2\bin\vivado.bat')
Prepending following Xilinx Vivado path(s) to the system path:
D:\vitis\Vivado\2019.2\bin
```

Hình 59 Khi nhập đường dẫn thành công

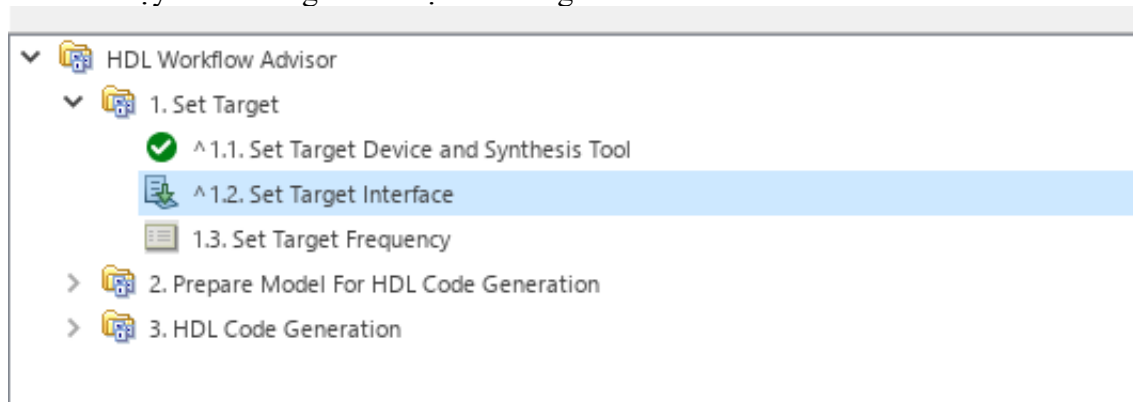
Bước 11: Nhấn Run This Task

Nếu lỗi hãy nhấn **turn on "Treat as atomic unit"** trong thông báo lỗi

Passed Set Target Device and Synthesis Tool.

Hình 60 Chạy thành công

Bước 12: Sao khi chạy thành công > Ta chọn Set Target Interface



Hình 61 Set Target Interface

Bước 13: Tại bảng 1.2 này phần Processor/FPGA synchronization chọn **Coprocessing-blocking**. Sau khi chọn các giá trị còn lại sẽ tự động set up

1.2. Set Target Interface

Analysis (^Triggers Update Diagram)

Set target interface for HDL code generation

Input Parameters

Processor/FPGA synchronization: **Coprocessing - blocking**

☐ Enable HDL DUT output port generation for test points

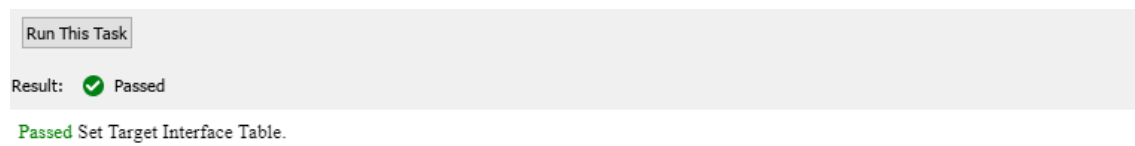
☒ Generate default AXI4 slave interface

Target platform interface table

Port Name	Port Type	Data Type	Target Platform Interfaces	Interface Mapping	Interface Options
x(k)	Inport	sfix16_E...	AXI4-Lite	x"100"	Options...
d(k)	Inport	sfix16_E...	AXI4-Lite	x"104"	Options...
e(k)	Output	sfix16_E...	AXI4-Lite	x"108"	

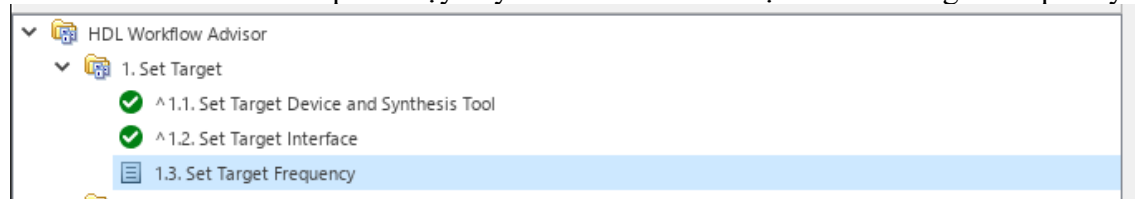
Hình 62 Set Target Interface

Bước 14: Run This Task



Hình 63 Run thành công

Bước 15: Khác với Matlab 2014 ta phải chạy đầy đủ các bước. Ta chọn 1.3 Set Target Frequency



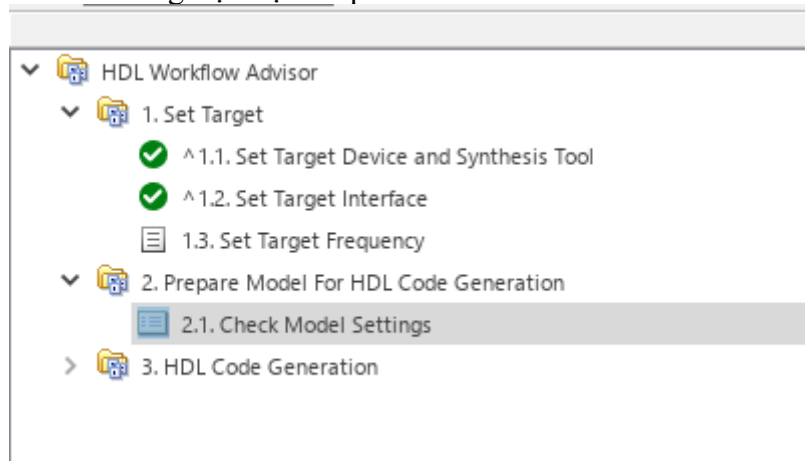
Hình 64 Set Target Frequency

Bước 16: Để thông số mặc định



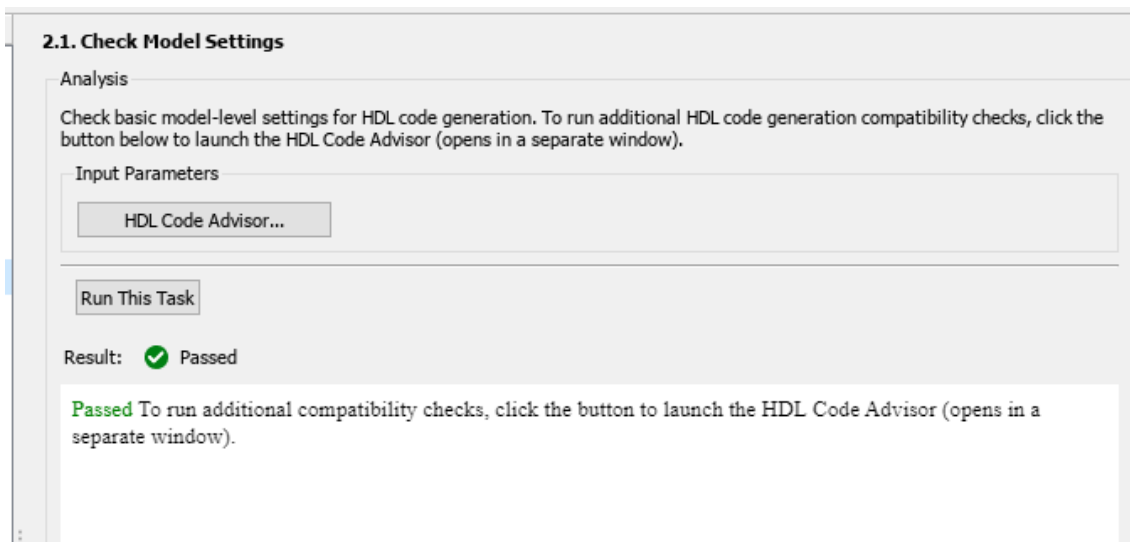
Hình 65 Bảng chọn Frequency

Bước 17: Chọn Check Model Settings tại mục Prepare Model For HDL Code Generation



Hình 66 Check Model Settings

Bước 18: Tại bảng Check Model Settings> Run This Task



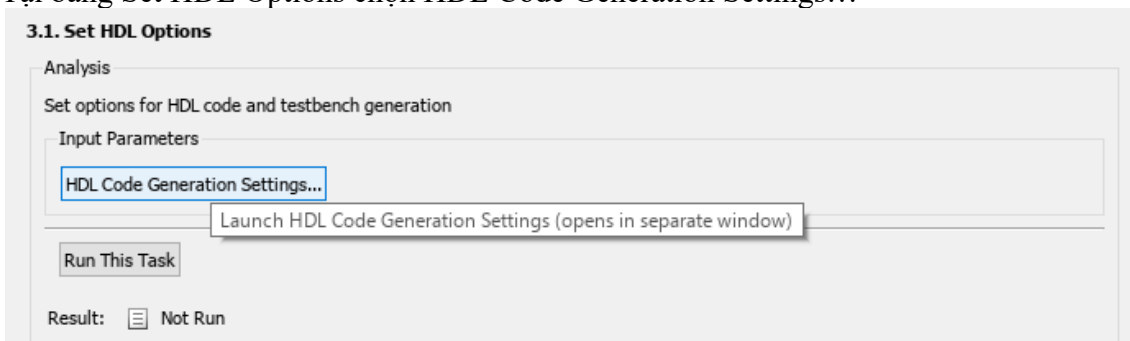
Hình 67 Sau khi check lỗi

Bước 19: Tại mục HDL Code Generation chọn phần 3.1 Set HDL Options để thiết lập các thông số trước khi generate Code

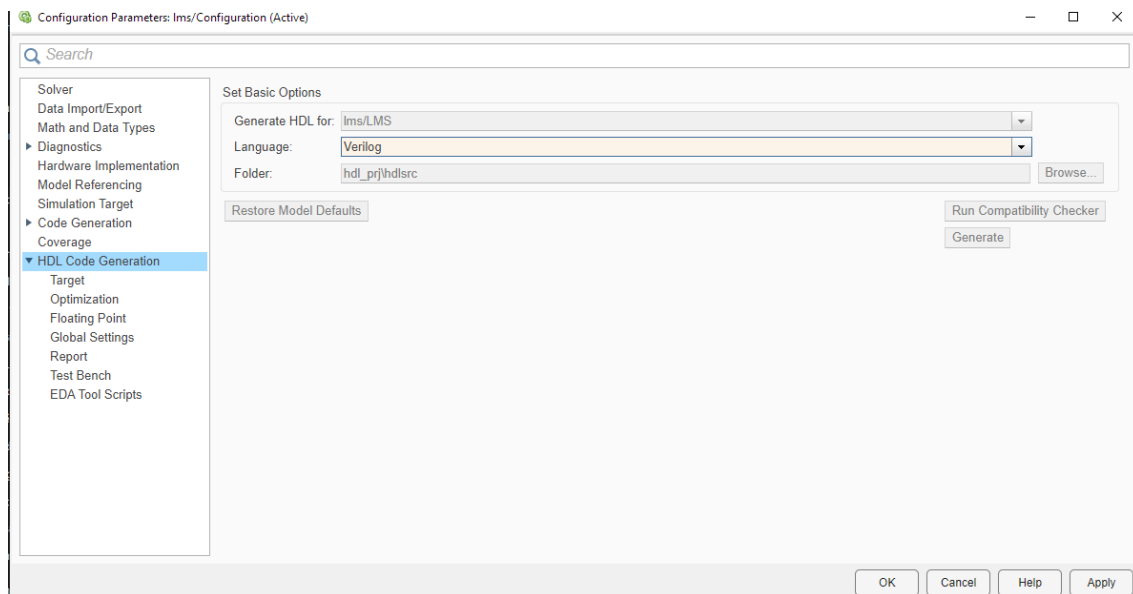


Hình 68 Set HDL Options

Bước 20: Tại bảng Set HDL Options chọn HDL Code Generation Settings...

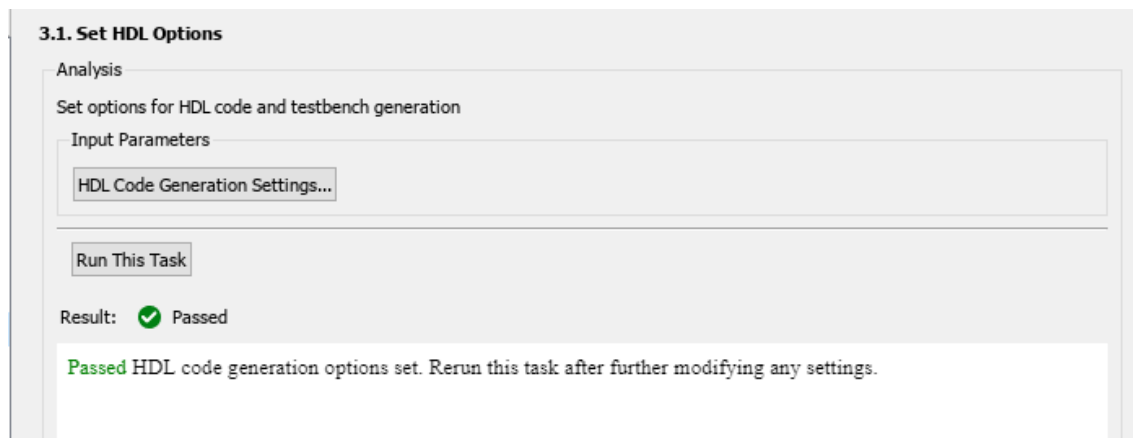


Hình 69 Bảng Set HDL Options



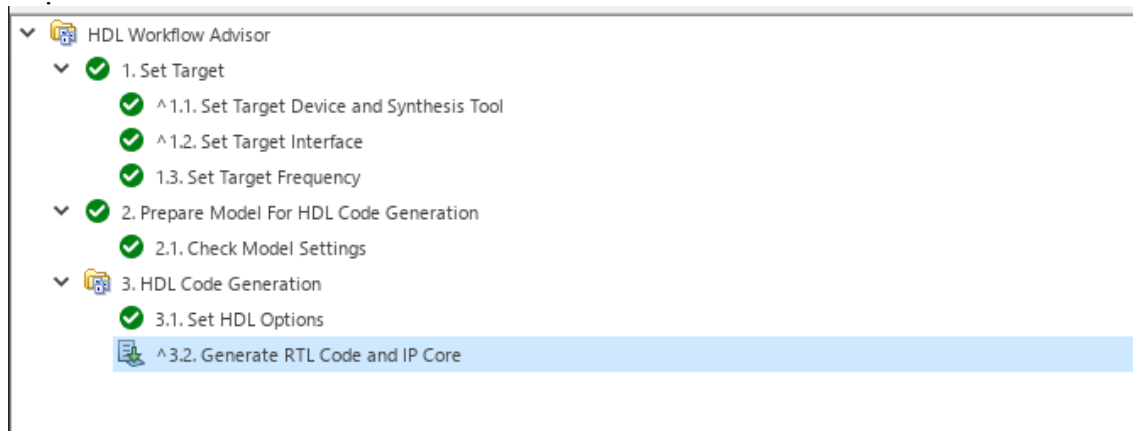
Hình 70 Bảng setting

Bước 21: Sau khi đã thiết lập xong quay lại Set HDL Options > Run This Task



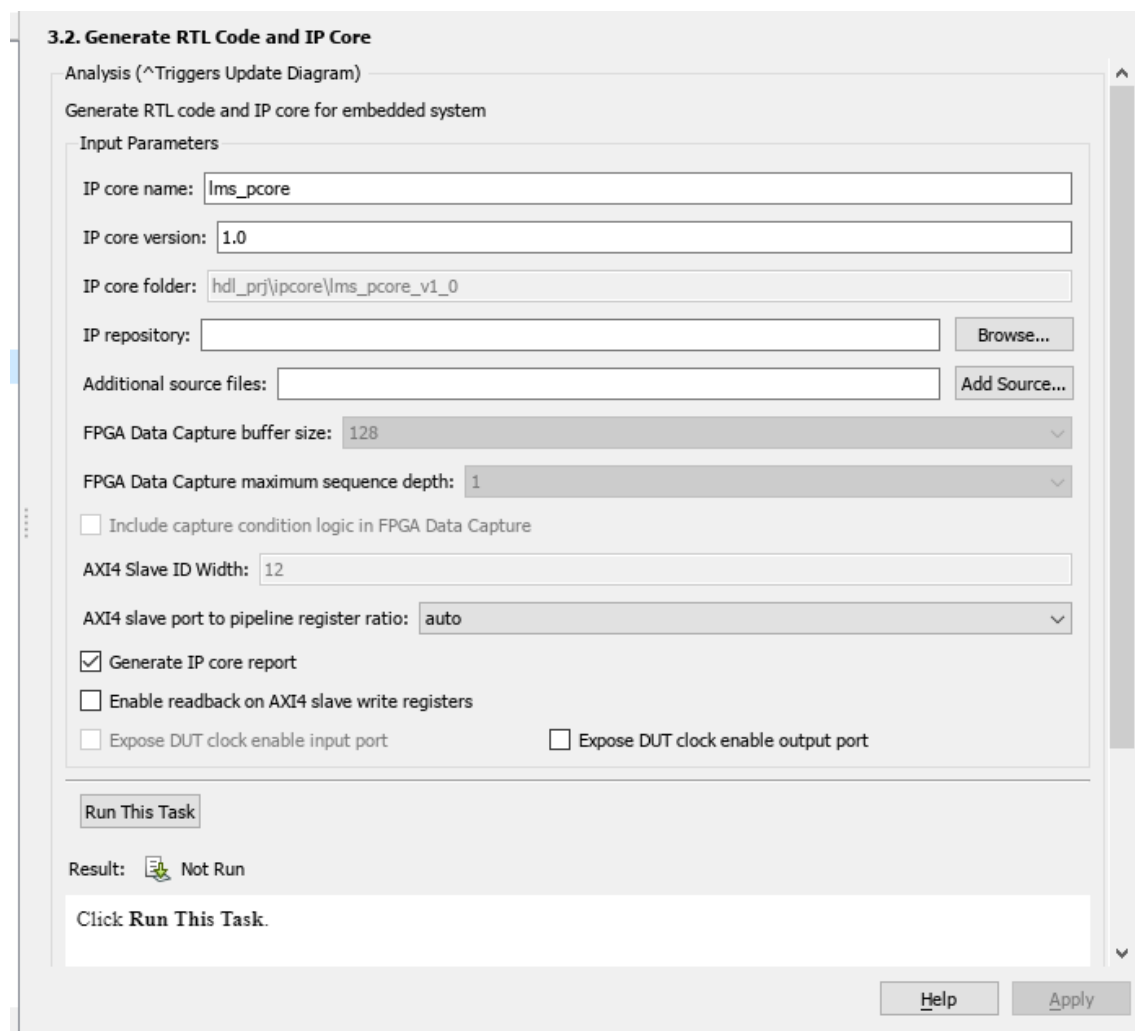
Hình 71 Run thành công

Bước 22: Chọn Generate RTL Code and IP Core



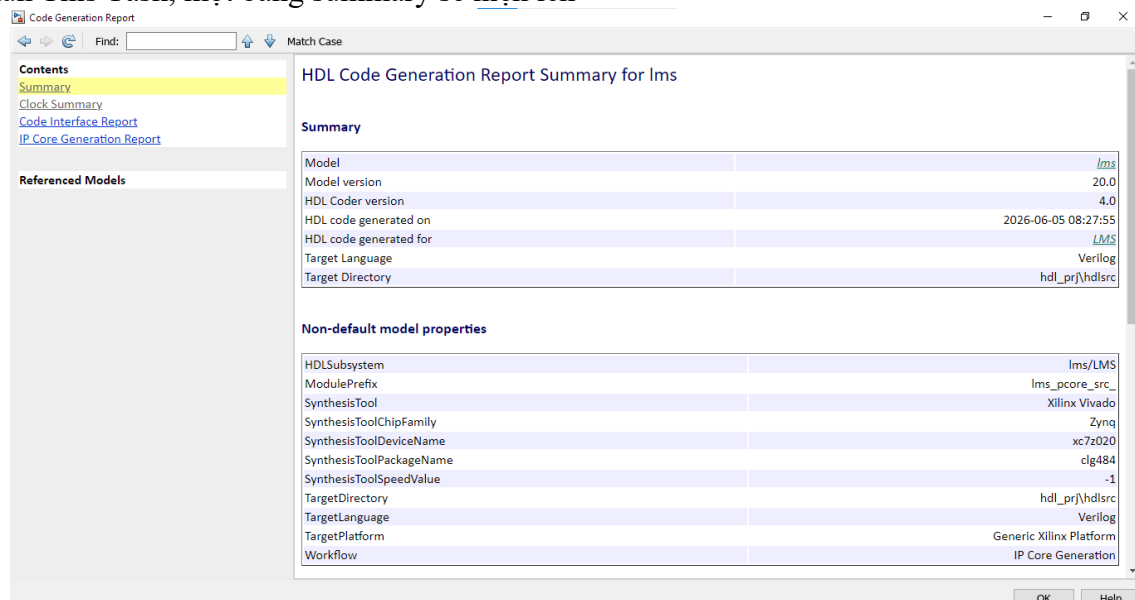
Hình 72 Generate RTL Code and IP Core

Bước 23: Tại bảng Generate RTL Code and IP Core đặt tên file là ***lms_pcore***



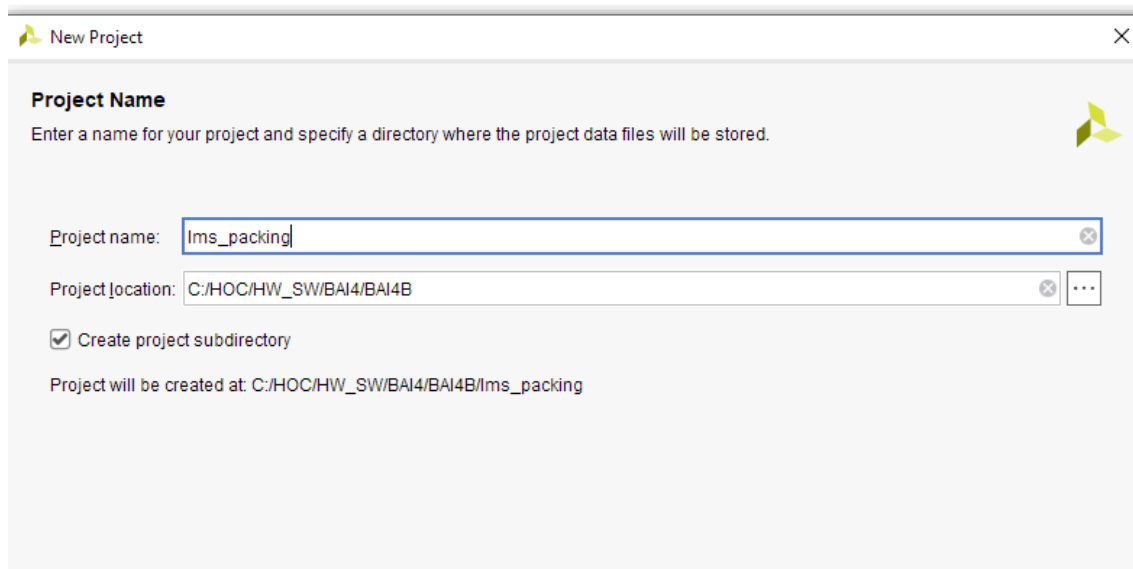
Hình 73 Tạo code

Sau khi Run This Task, một bảng summary sẽ hiện lên



Hình 74 Summary

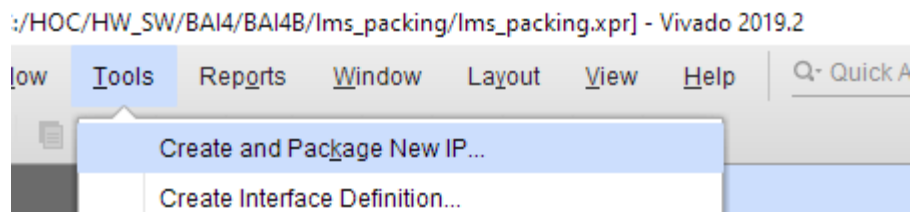
Bước 24: Mở Vivado tạo project tên ***lms_packing***



Hình 75 Project mới

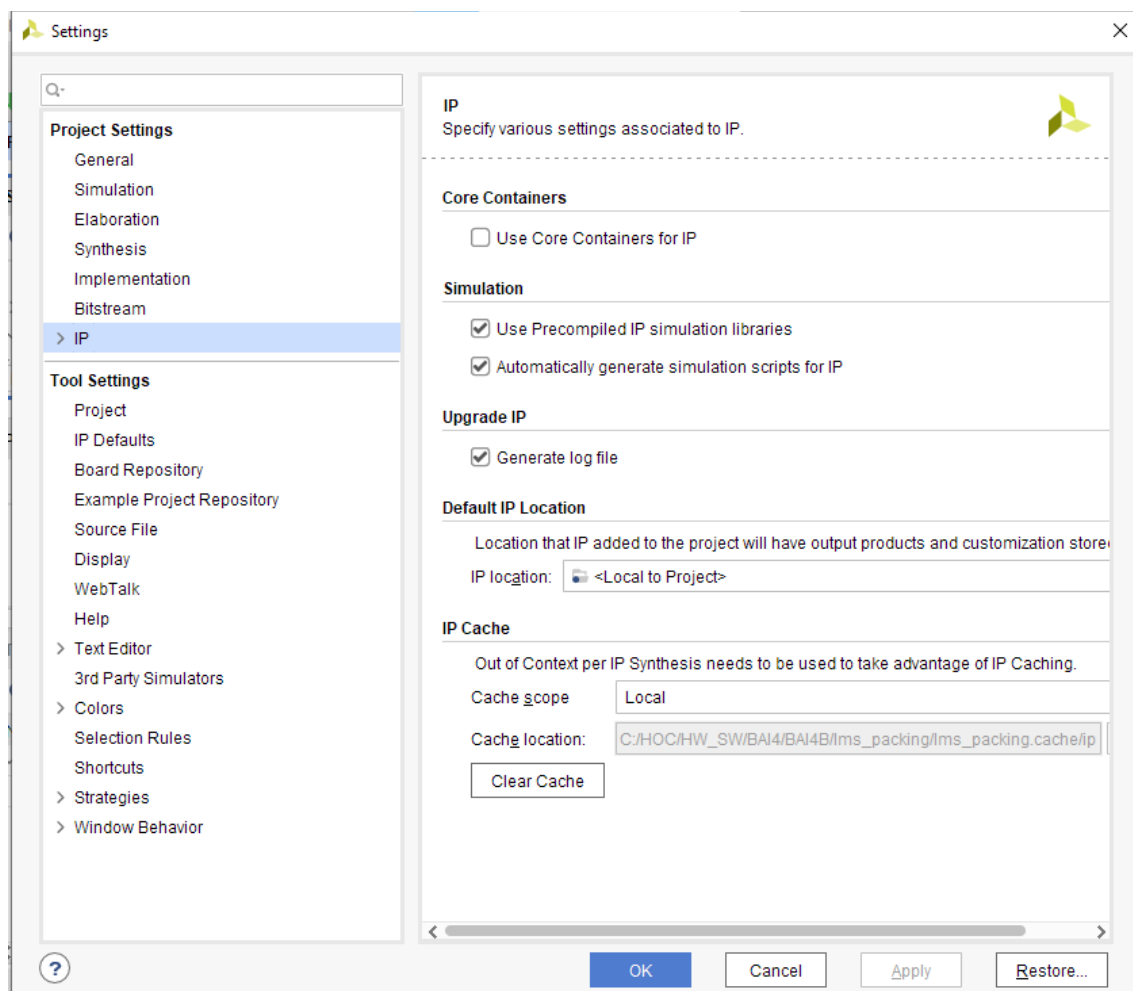
Thiết lập project mới như 4A a

Bước 25: Chọn Tool> Settings



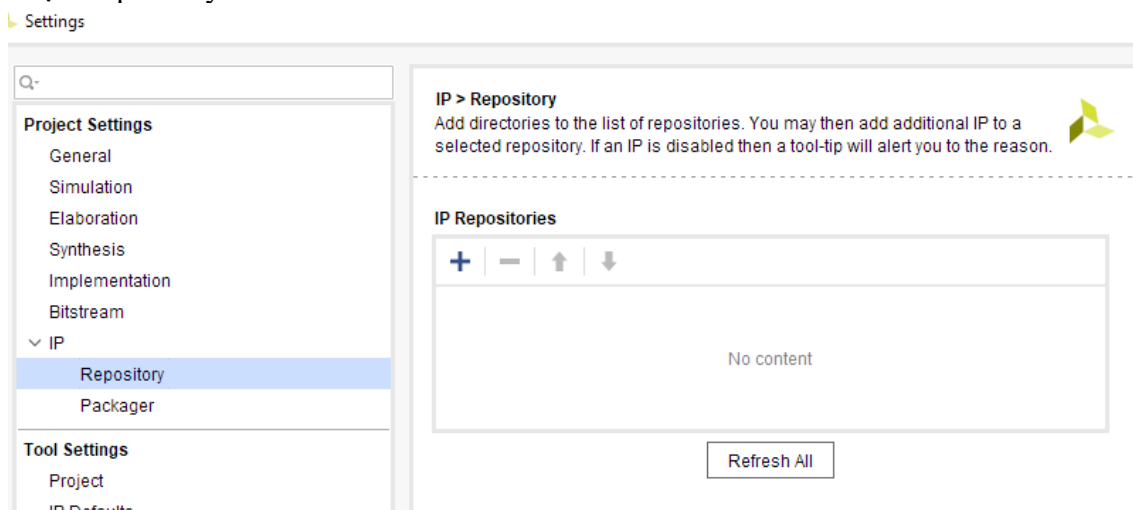
Hình 76 Bảng chọn Tools

Bước 26: Chọn IP



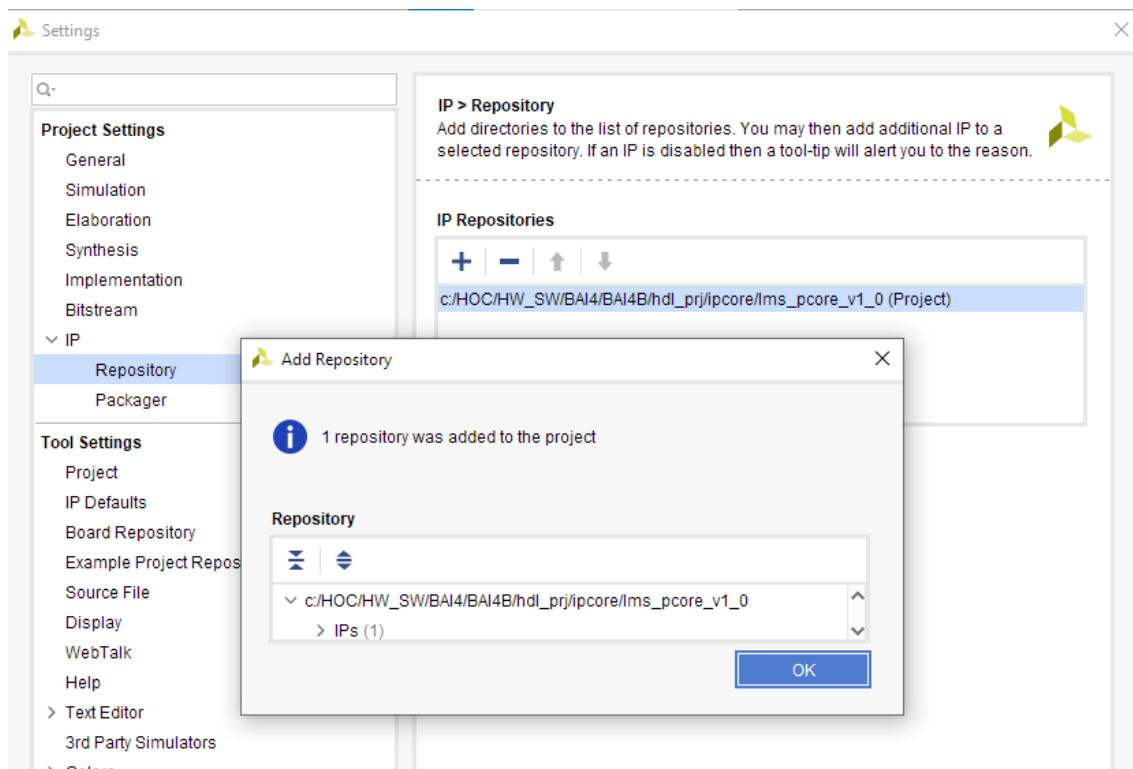
Hình 77 Bảng setting

Bước 27: Chọn Repository

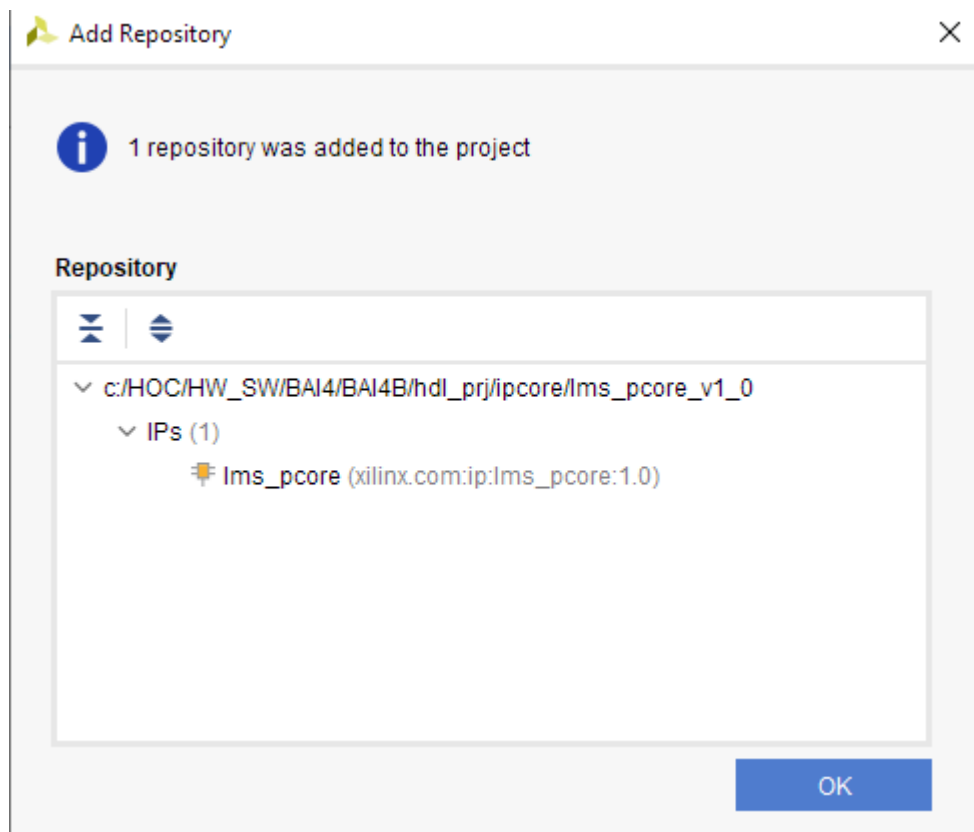


Hình 78 IP>Repository

Bước 28: Nhấn dấu cộng > Nhập đường dẫn IP đã có:

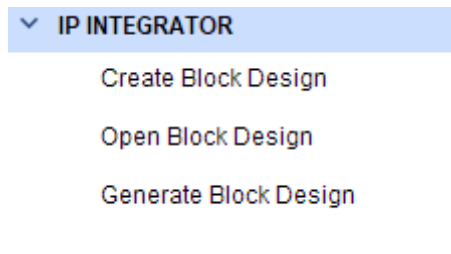


Hình 79 Add IP



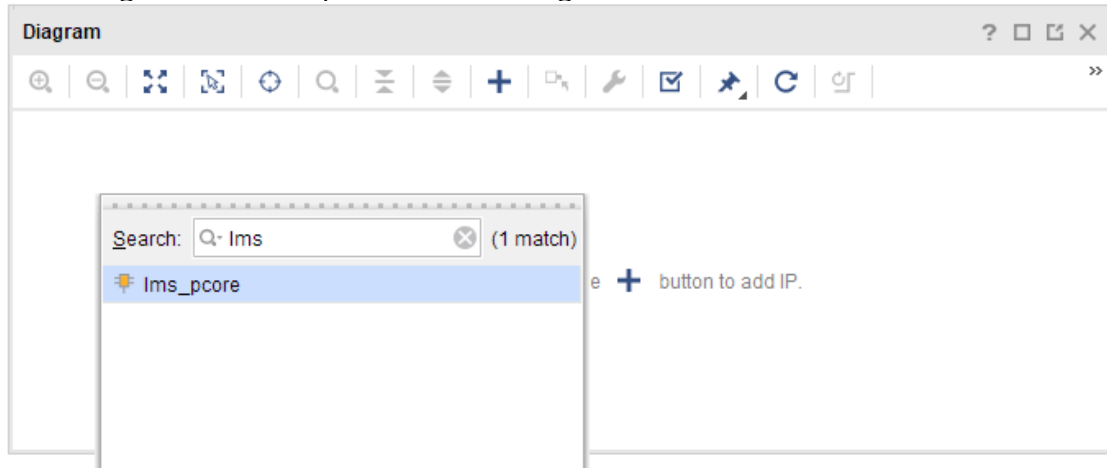
Hình 80 IP đã tạo từ matlab

Bước 29: Tại IP INERGATOR > Create Block Design

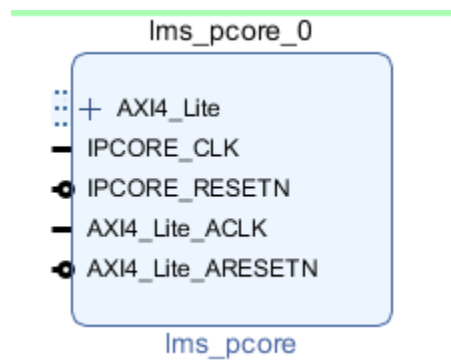


Hình 81 IP integrator

Bước 30: Add IP > gõ lms, có lms_pcore là thành công



Hình 82 Add IP đã có



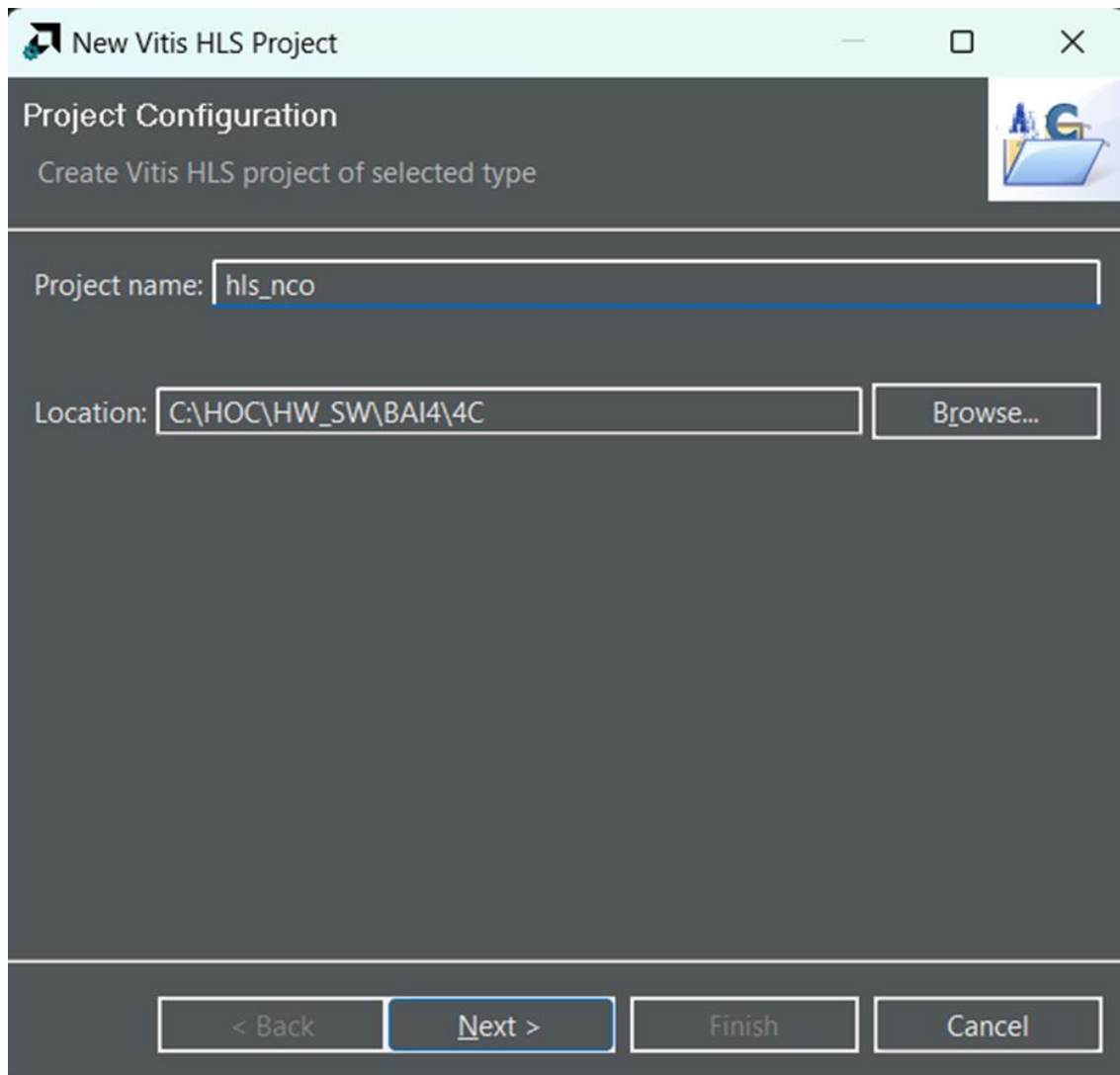
Hình 83 Khối IP được tạo ra

Exercise 4C: Creating IP in Vivado HLS

Bước 1: Tạo project trên Vitis HLS

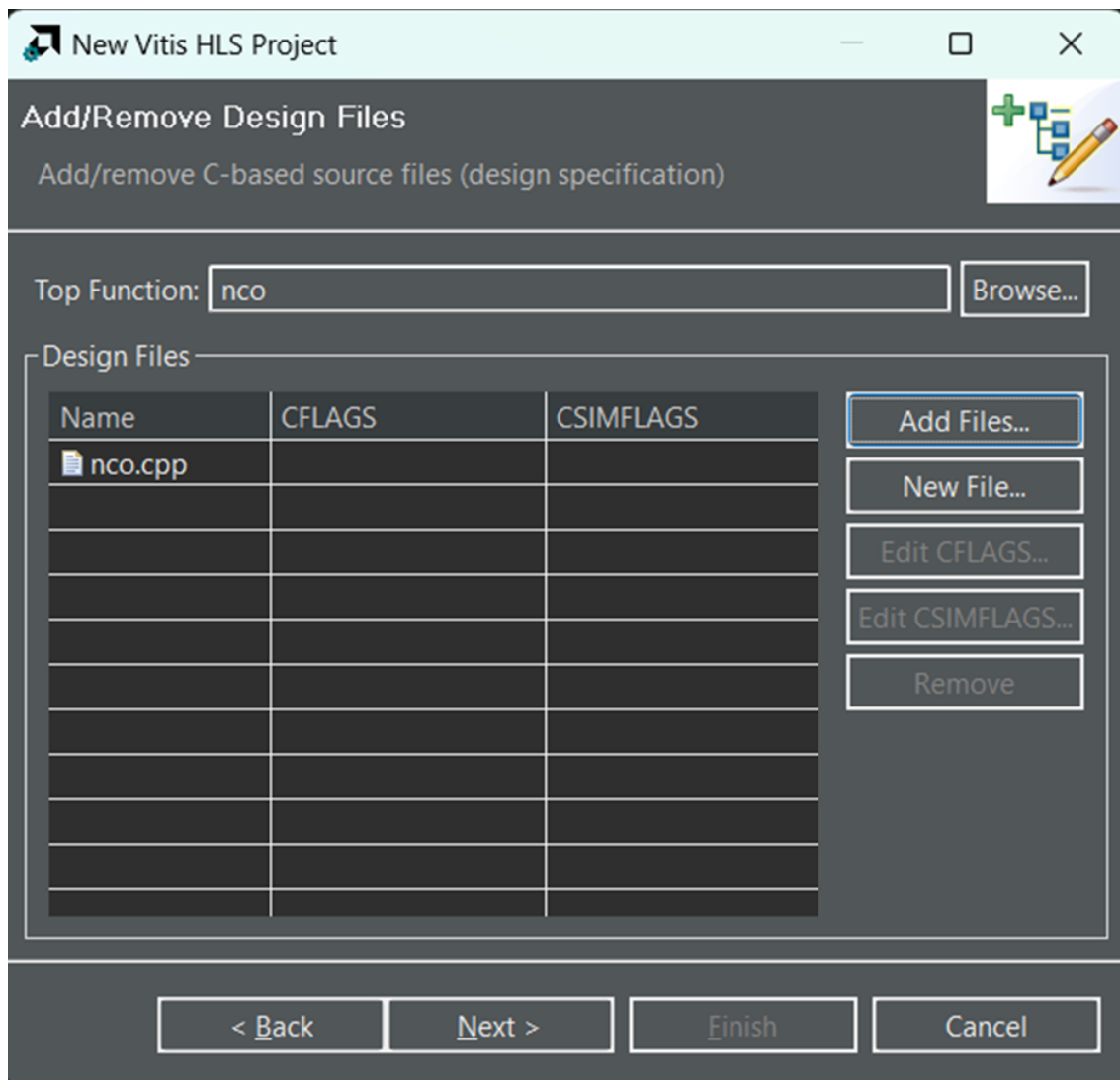
Các bước tạo project trên Vitis HLS tương tự như những lần trước đó nhưng lần này, ta sẽ thêm source có sẵn trong bước tạo project.

1a. Mở Vitis HLS và tạo project mới, đặt tên và chọn đường dẫn lưu cho project như hình:



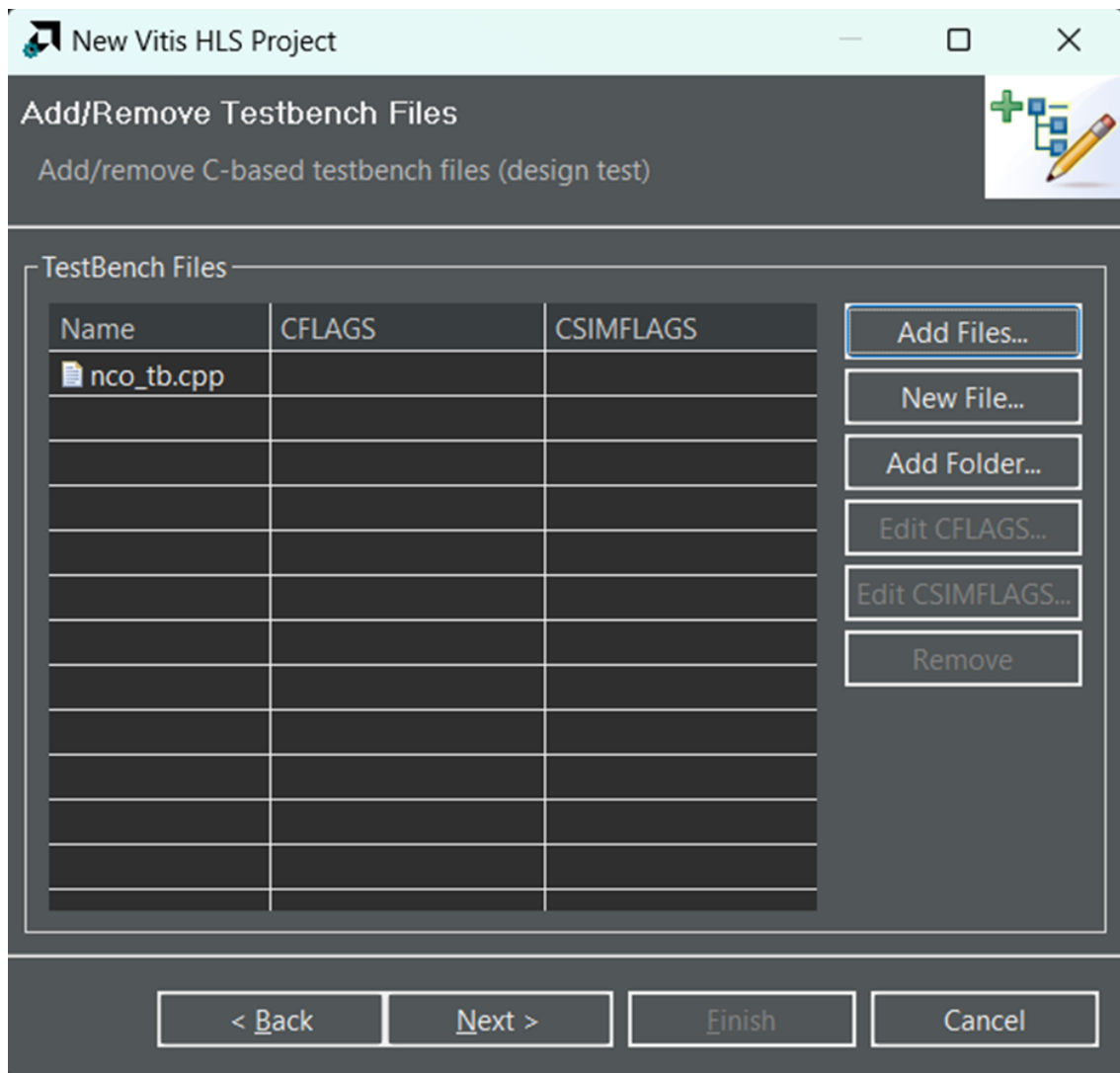
Hình 84 Tạo project Vitis HLS 1

1b. Đặt tên cho Top Function và thêm Source cho khối NCO. Ở cửa sổ này, chọn “Add Files” rồi tìm đến file nco.cpp rồi thme vào project, sau đó nhấn “Next”.



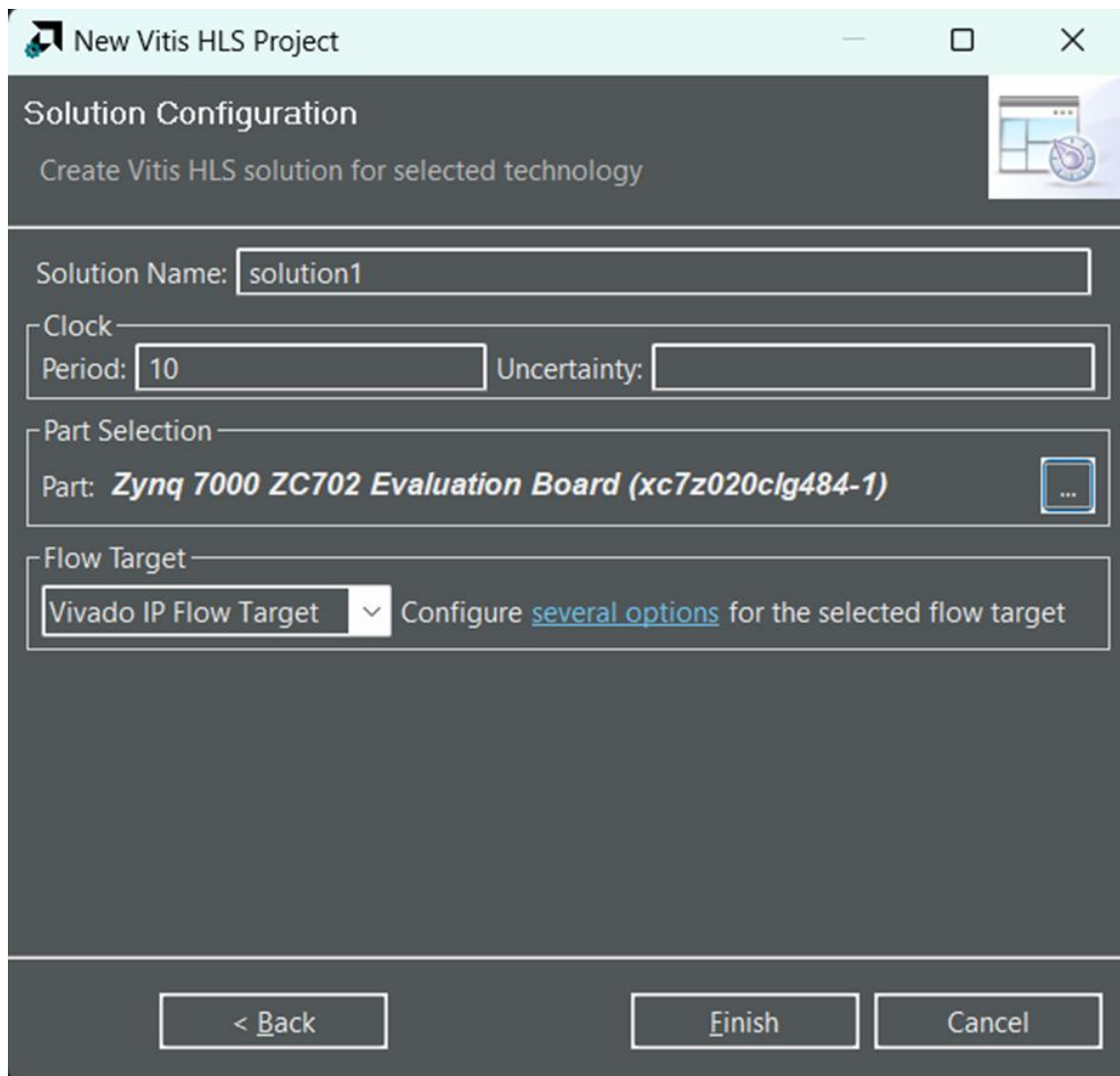
Hình 85 Tạo project Vitis HLS 2

1c. Sau khi đã thêm file Source cho khối NCO, ta cần phải thêm file Testbench để kiểm chứng chức năng của nó. Nhấn “Add Files” rồi tìm đến file nco_tb.cpp rồi thêm vào sau đó nhấn “Next”.



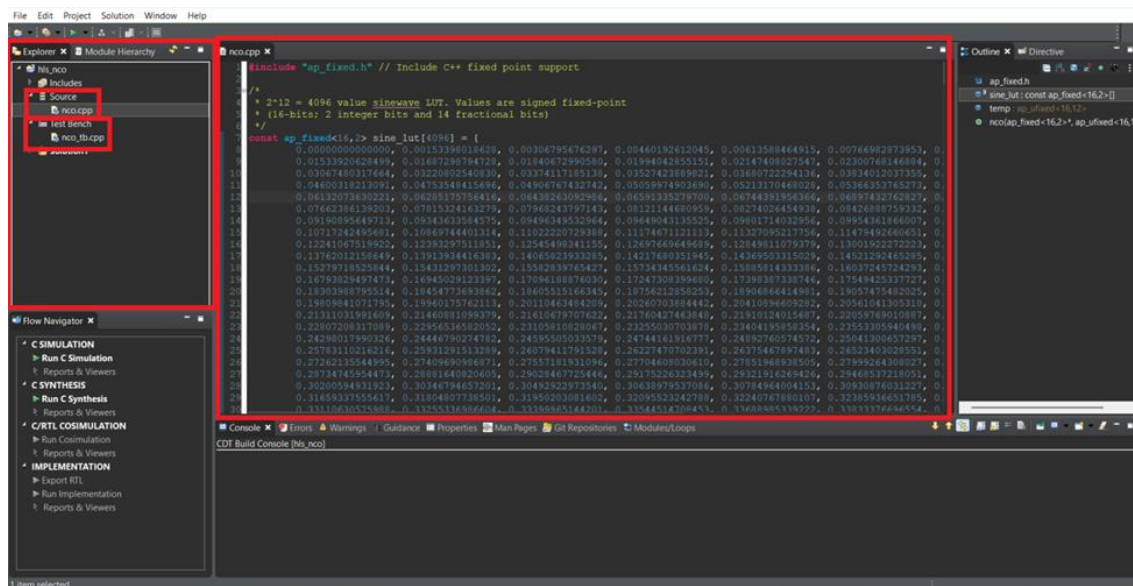
Hình 86 Tạo project Vitis HLS 3

1d. Tiến hành chọn Board cho project rồi đặt tên và cấu hình cho solution1.



Hình 87 Tạo project Vitis HLS 4

1e. Sau khi Vitis HLS đã tạo project xong màn hình GUI sẽ xuất hiện. Kiểm tra xem quá trình tạo project có thành công không bằng các kiểm tra “Source” và “Testbench” trong cửa sổ Explorer. Nếu 2 file đã thêm có xuất hiện tại đúng vị trí thì có thể mở 2 file đó lên, trong cửa sổ chính sẽ xuất hiện nội dung code của 2 file đó như hình.



Hình 88 Tạo project Vitis HLS 5

Bước 2: Kiểm tra source và kết quả C Simulation

Hàm nco.cpp:

```
#include "ap_fixed.h"
const ap_fixed<16,2> sine_lut[4096] = { ... }
ap_ufixed<16,12> temp = 0.0;

void nco (ap_fixed<16,2> *sine_sample, ap_ufixed<16,12> step_size){
    ap_ufixed<12,12> address;
    temp+=step_size;
    address = ap_ufixed<12,12>(temp);
    *sine_sample = sine_lut[(int)address];
}
```

Giải thích:

Thay vì thực hiện việc tính toán ra biên độ của hàm sin, IP này sẽ sử dụng một LUT để lưu các giá trị đã được tính sẵn và khi đưa tham số vào, IP này sẽ truy cập đến phần tử tương đương trong LUT và trả về giá trị đó như các truy cập bộ nhớ.

Code sử dụng thêm thư viện ap_fixed để hỗ trợ kiểu dữ liệu dấu chấm thập phân dùng cho giá trị analog của NCO. LUT chứa các giá trị sin đã tính sẵn được lưu trong một mảng ap_fixed<16,2> với kích thước mảng là 4096. ap_fixed<16,2> là kiểu dữ liệu 16b với 2 bit int gồm 1 bit dấu dương hoặc âm và 1 bit 0 hoặc 1 để tạo ra phần nguyên có thể biểu diễn các số -1, 0 và 1. 14 bit còn lại là phần thập phân để tạo phần thập phân có giá trị từ 0 đến 0.99993896484375. Kết hợp 2 phần này tạo thành một giá trị có thể biểu diễn biên độ analog của sóng sin với độ phân giải cao.

Bên cạnh ap_fixed<16,2>, code này còn sử dụng ap_ufixed<16,12> để lưu và biểu diễn một kiểu dữ liệu 16b với 12b phần nguyên không dấu và 4b phần thập phân. Vì LUT chứa 4096 giá trị nên mảng tương ứng cần phải có $2^{12} = 4096$ phần tử và biến dùng để truy cập vào mảng này cũng cần là kiểu dữ liệu bằng hoặc lớn hơn 12b và ở trong code này là ap_ufixed<16,12> temp.

Hàm nco() là phần để synthesis ra được khối phần cứng thực hiện việc trả về một giá trị sóng sin mà đã được lưu trước đó trong LUT. Hàm nhận 2 parameter là con trỏ đến địa chỉ lưu giá trị sóng sin và bước nhảy để tính ra phần tử trong LUT cần truy cập. Hàm bắt đầu với việc khai báo một biến 12b để làm địa chỉ truy cập mảng LUT rồi tính ra địa chỉ cần truy cập từ bước nhảy và địa chỉ truy cập trước đó. Sau đó, hàm ép biến temp về kiểu dữ liệu 12b rồi gán cho biến address dùng để truy cập vào mảng LUT rồi ghi vào nơi con trỏ đang chỉ đến.

File testbench khởi tạo giá trị bước pha STEP = 20.79 rồi gọi hàm nco() 1024 lần để tạo ra 1024 mẫu, mỗi mẫu được ghi vào file 'nco_sine.m' để phân tích trong MATLAB. Vì STEP = 20.79 nên số mẫu cần thiết để hoàn thành một chu kỳ sóng sin là 197 mẫu. Do đó, tập dữ liệu gồm 1024 mẫu được tạo bởi testbench chứa khoảng 5 chu kỳ sóng sin.

2a. Tiến hành chạy C Simulation để kiểm chứng chức năng của code C chạy đúng. Sau khi chạy xong, kết quả nhận được trong log sẽ như sau:

```
INFO: [SIM 2] ***** CSIM start *****
INFO: [SIM 4] CSIM will launch GCC as the compiler.
    Compiling ../../../../Users/PC/Downloads/nco_tb.cpp in debug mode
    Compiling ../../../../Users/PC/Downloads/nco.cpp in debug mode
    Generating csim.exe
File open for writing.
```

Sample output to file complete.

INFO: [SIM 1] CSim done with 0 errors.

INFO: [SIM 3] ***** CSIM finish *****

Khi đã chạy xong C Simulation, file chứa các mẫu của sóng sin sẽ được Vitis HLS lưu lại trong đường dẫn đã được khai báo trong file testbench. Ví dụ nếu trong file testbench đặt đường dẫn char *outfile = "E:\\nco_sine.m"; thì kết quả sẽ được lưu tại ổ đĩa E.

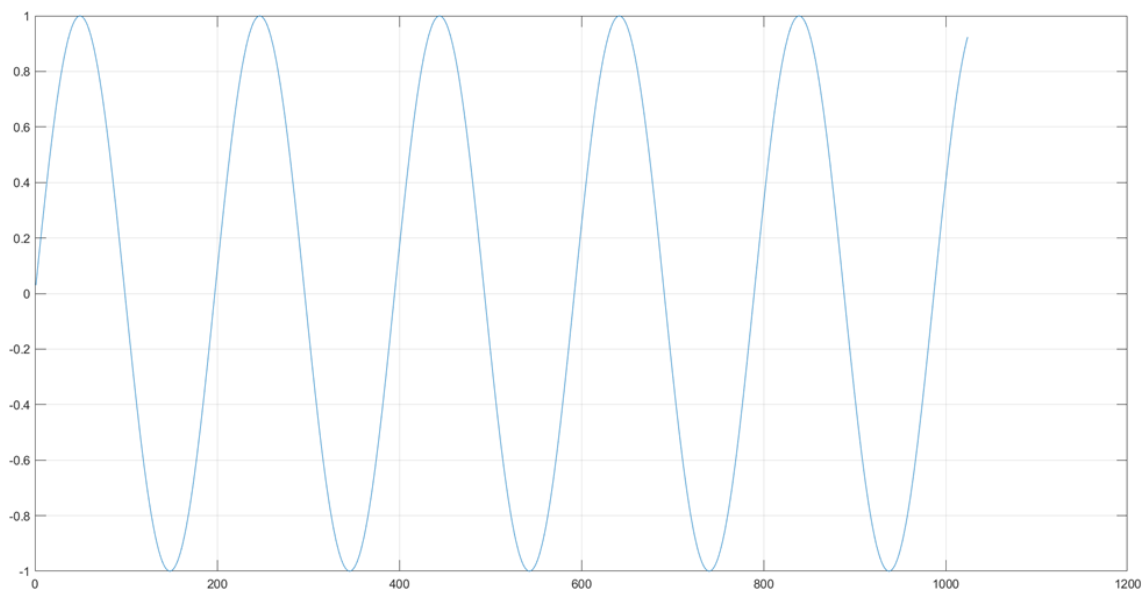
2b. Từ file này ta có thể tiến hành mở và kiểm tra kết quả trong MATLAB để xem tín hiệu sóng sin được tạo ra bởi khối NCO có đúng kết quả mong đợi hay chưa. File “nco_sine.m” được tạo bởi Vitis HLS như sau:

```
nco_sine = [  
0.03063964843750,  
0.06280517578125,  
0.09490966796875,  
...  
0.91107177734375,  
0.92382812500000,  
];
```

Để kiểm chứng và quan sát kết quả này, ta thêm các dòng sau vào cuối:

```
plot(nco)  
grid on
```

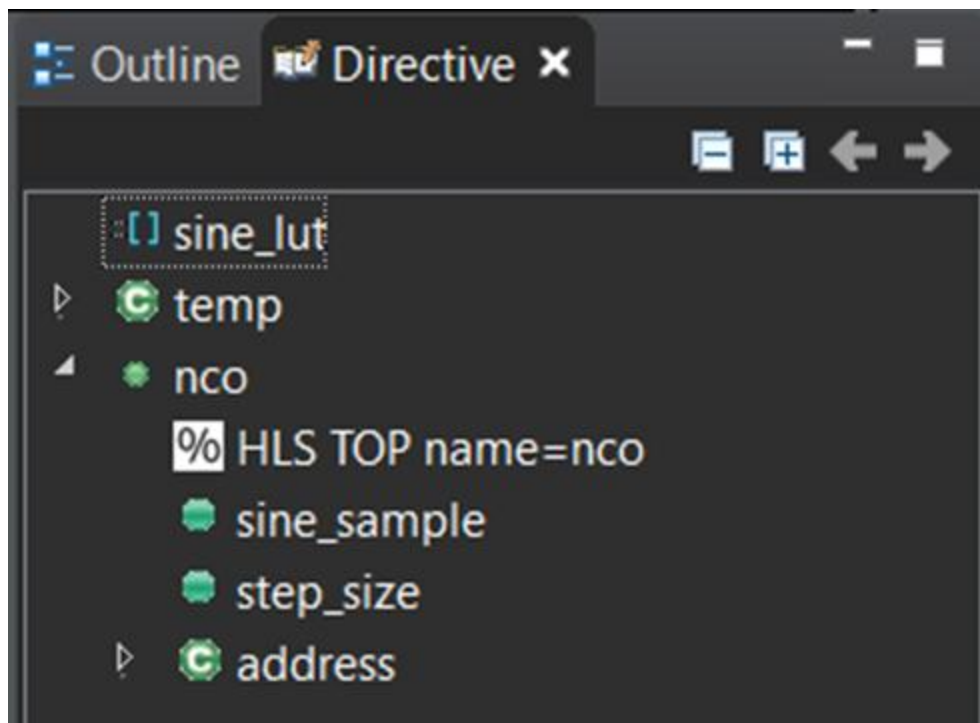
Ngoài ra cần phải đổi tên vector từ nco_sine thành nco. Sau đó tiến hành chạy file và đồ thị sau sẽ xuất hiện:



Hình 89 Kết quả plot output khối NCO trên MATLAB

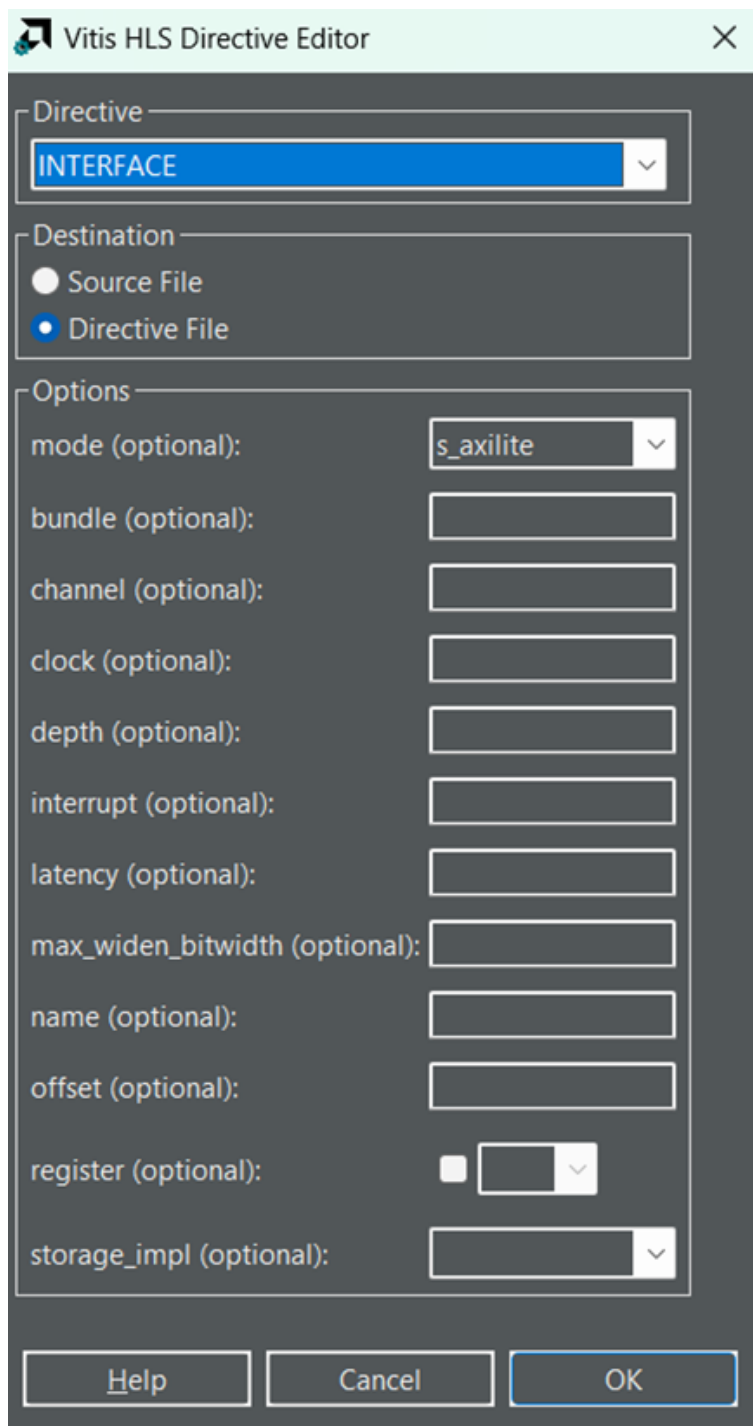
Đây là output của khối NCO, như trong hình sóng sine dao động trong biên độ từ 1 đến -1 với mẫu đầu tiên có giá trị là 0 tương ứng với LUT của NCO. Sóng gồm 5 chu kỳ với mỗi chu kỳ gồm 197 mẫu đúng như kết quả dự đoán.

Bước 3: tiếp tục thêm AXI interface cho khối NCO bằng cách chọn vào file nco.cpp và chọn tab Directive để cho NCO làm AXI-Lite Slave.



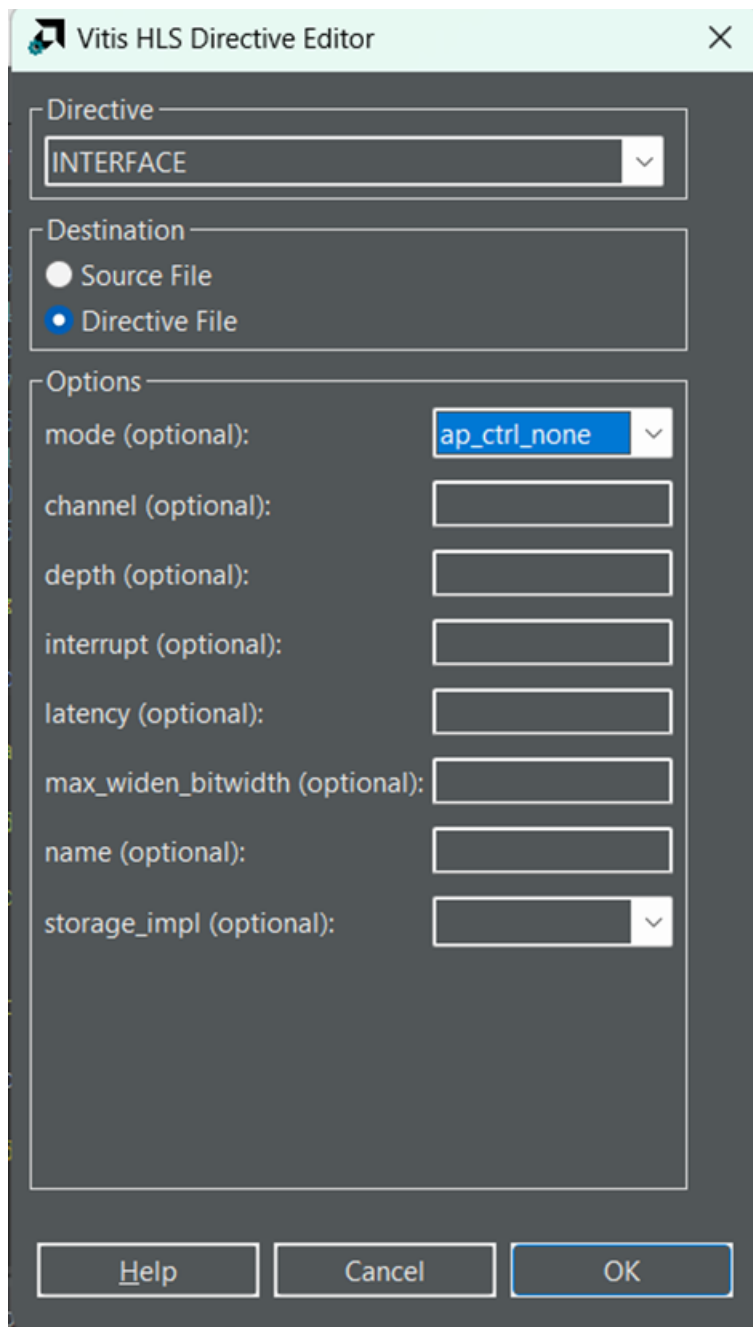
Hình 90 Cửa sổ Directive

3a. Thêm directive Interface và mode s_axilite cho nco, các lựa chọn khác để nguyên rồi nhấn “OK”



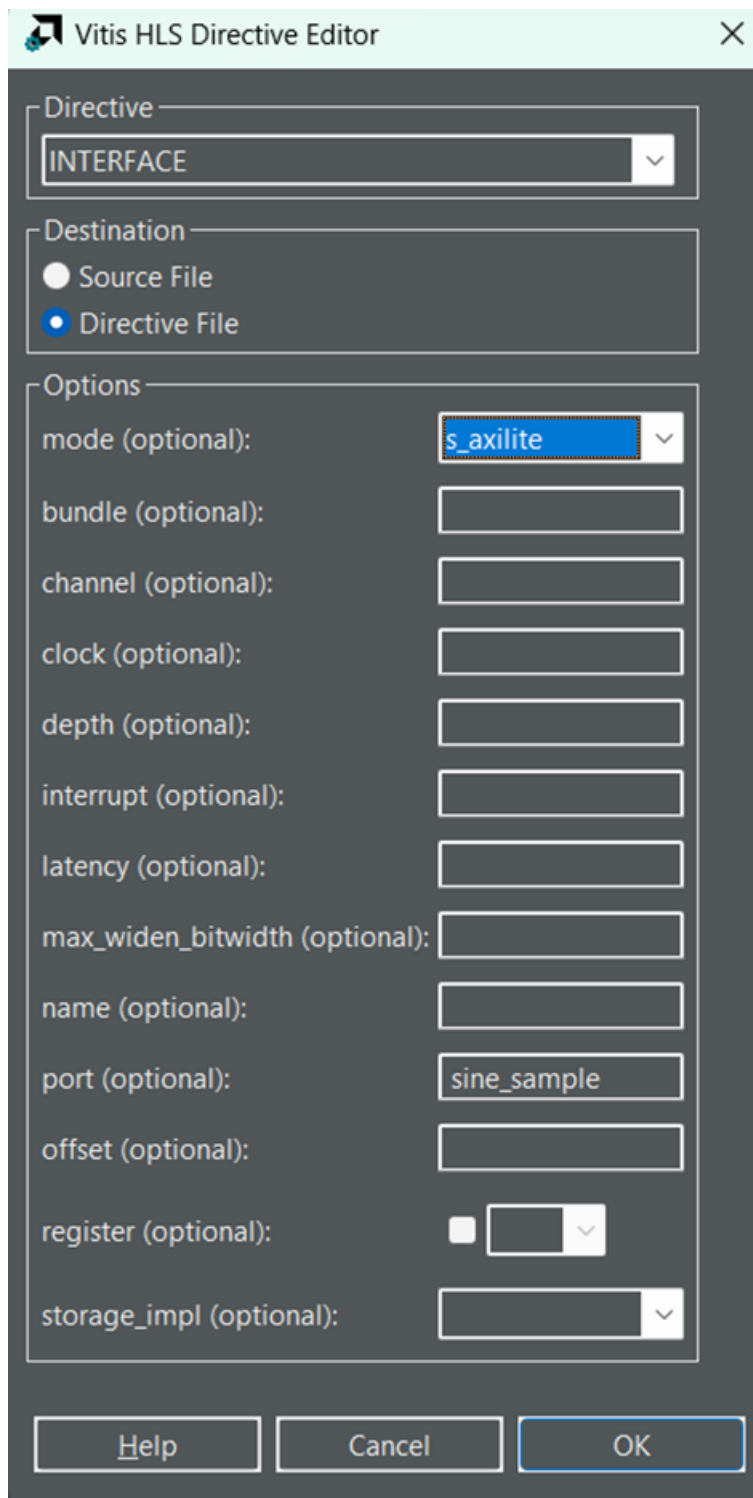
Hình 91 Thêm interface s_axilite cho nco

3b. Thêm directive Interface và mode ap_ctrl_none cho nco, các lựa chọn khác để nguyên rồi nhấn “OK”



Hình 92 Thêm interface *ap_ctrl_none* cho *nco*

3c. Thêm directive Interface và mode *s_axilite* cho *sine_sample*, các lựa chọn khác để nguyên rồi nhấn “OK”



Hình 93 Thêm interface cho sine_sample

3d. Thêm directive Interface và mode s_axilite cho step_size, các lựa chọn khác để nguyên rồi nhấn “OK”

Vitis HLS Directive Editor

Directive

INTERFACE

Destination

☐ Source File

☒ Directive File

Options

mode (optional): s_axilite

bundle (optional):

channel (optional):

clock (optional):

depth (optional):

interrupt (optional):

latency (optional):

max_widen_bitwidth (optional):

name (optional):

port (optional): step_size

offset (optional):

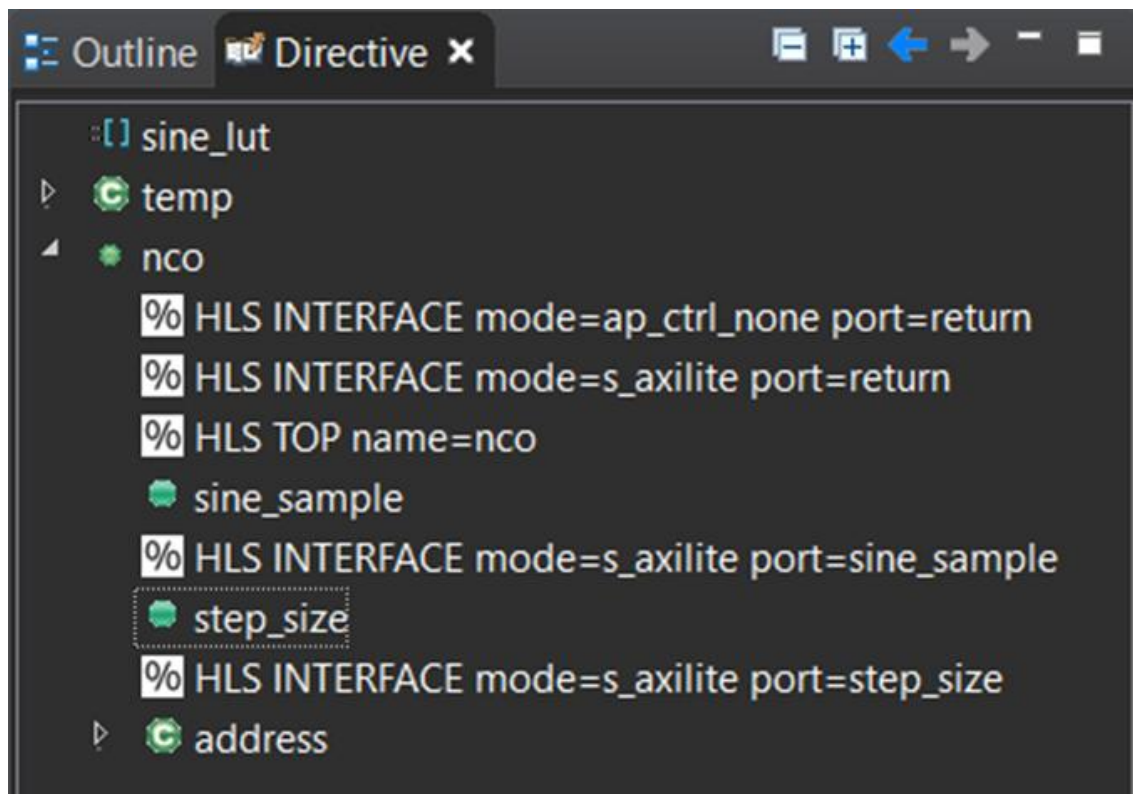
register (optional): ☐

storage_impl (optional):

Help Cancel OK

Hình 94 Thêm interface cho step_size

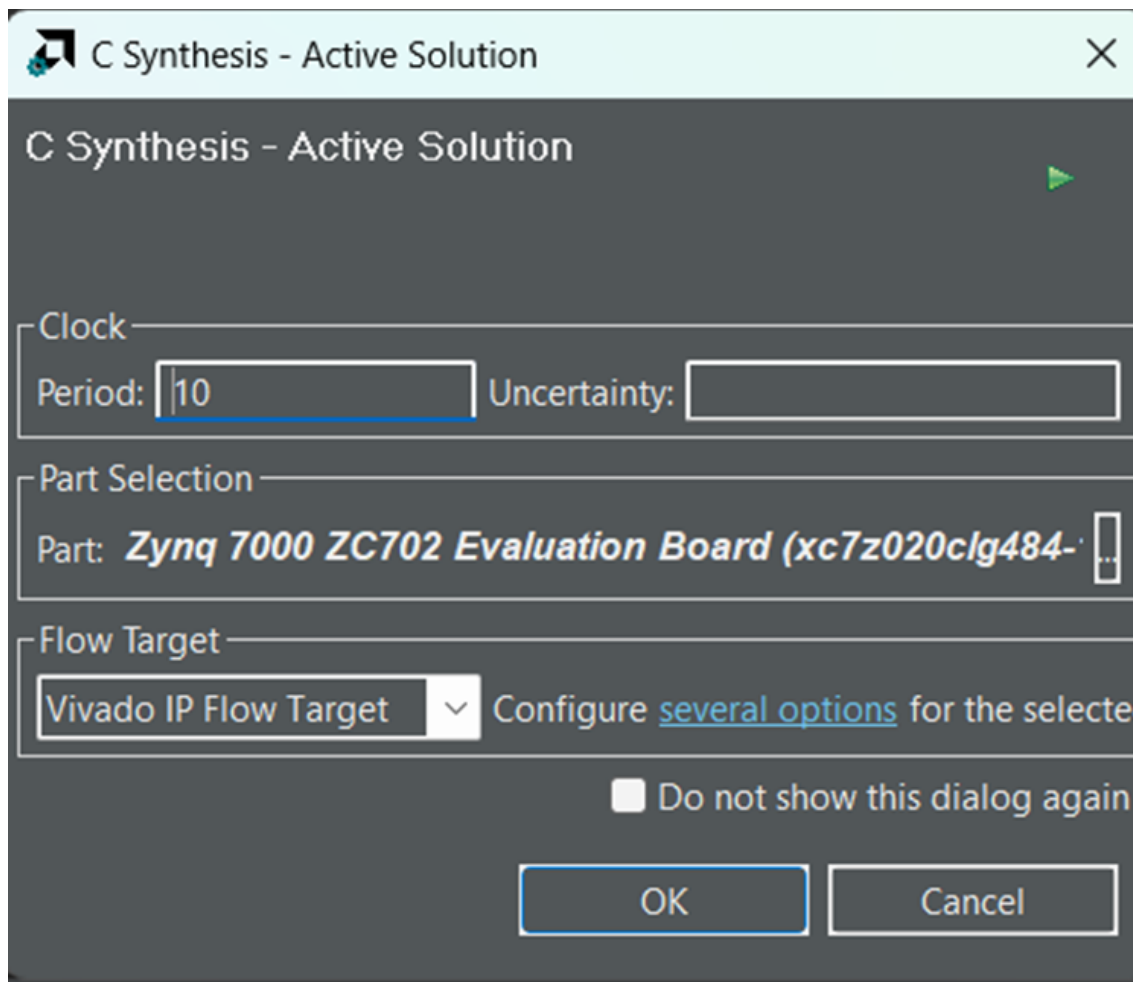
Sau khi đã thêm 4 directive trên, kiểm tra lại kết quả trong cửa sổ Directive, đảm bảo kết quả đúng như yêu cầu



Hình 95 Cửa sổ Directive sau khi đã thêm giao diện AXI-Lite

Bước 4: Chạy HLS và xuất file HDL

4a. Chạy quá trình synthesis bằng cách chọn Run C Synthesis trong cửa sổ Flow Navigator. Đây là cách ta tạo ra code HDL cho khối từ source C. Cửa sổ Act0ive Solution sẽ hiện lên rồi chọn period = 10.



Hình 96 Cửa sổ C synthesis

Synthesis Summary Report of 'nco'

General Information

Date: Mon Jun 1 14:45:43 2026

Version: 2023.2 (Build 4023990 on Oct 11 2023)

Project: hls_nco

Solution: solution1 (Vivado IP Flow Target)

Product family: zynq

Target device: xc7z020-clg484-1

Timing Estimate

Target

Estimated

Uncertainty

10.00 ns

6.331 ns

2.70 ns

Performance & Resource Estimates

Modules

Loops

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM	
nco				-	1	10.000		-	2	-	no	4	0	98	141	0

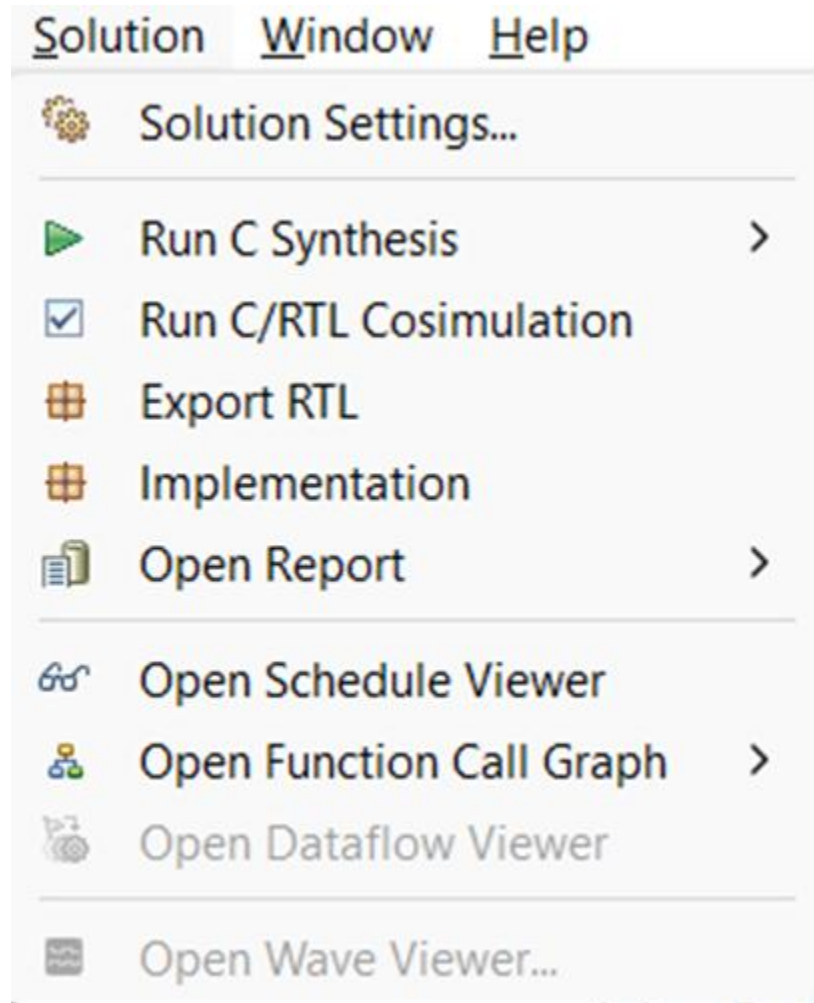
Hình 97 Report quá trình synthesis

Kết quả synthesis cho thấy thiết kế NCO đáp ứng tốt yêu cầu timing. Thời gian ước tính là 6.331 ns so với target 10.00 ns cho phép khối NCO chạy với tần số ổn định tối đa là 157.952 MHz. Latency của hàm nco chỉ là 1 chu kỳ tương ứng với 10 ns, tuy nhiên initiation interval là 2 chu kỳ nghĩa là mỗi 2 chu kỳ xung nhịp mới có thể nhận một đầu vào mới vì khối NCO không được áp dụng pipeline.

NCO sử dụng 141 LUT và 98 FF, không dùng DSP block nào do các phép tính cộng dồn đơn giản có thể thực hiện bằng logic và 4 BRAM được dùng để lưu LUT chứa giá trị sóng sin gồm 4096 giá trị $\text{ap_fixed} < 16,2 \text{ 16b}$, vì LUT của NCO có kích thước đủ lớn để Vitis HLS chọn BRAM. Nhìn chung thì đây là kết quả synthesis tốt và phù

hợp cho một NCO đơn giản dùng cho IP tích hợp trong hệ thống Zynq.

4b. Sau khi quá trình synthesis hoàn tất, ta cần phải xuất code HDL từ main toolbar. Chọn Solution > Export RTL như hình:



Hình 98 Lựa chọn xuất code RTL

4c. Một cửa sổ Export RTL sẽ xuất hiện, chọn Export Format là Vivado IP (.zip) rồi nhấn OK. Sau khi đã xuất xong, trong solution1/impl/ip sẽ chứa IP package đã tạo. Từ IP này, ta có thể thêm nó vào IP Integrator.

Export RTL

×

Export RTL as IP/XO

Export Format

Vivado IP (.zip)

▼

Output Location

C:/HOC/HW_SW/BAI4/4C

Browse...

IP OOC XDC File

Browse...

IP XDC File

Browse...

IP Configuration

Vendor

Library

Version

Description

Display Name

Taxonomy

☐ Do not show this dialog again

OK

Cancel

Hình 99 Cửa sổ Export RTL

Hình nhóm:

Điểm danh từ trái sang: (Ngày 1/6/2026)



Điểm danh từ trái sang(ngày 5/6/2026)

Thành Đô – Thái Khương – Quang Huy – Tuấn Kiệt – Phi Hùng



LINK GITHUB

https://github.com/LDTKhuong/LAB1_Nhom_15